

REMERCIEMENTS

Je tiens à exprimer ma profonde gratitude à Monsieur Noel Alain Olivier MEKULU MVONDO AKAME, Directeur Général de la Caisse Nationale de Prévoyance Sociale (CNPS), pour m'avoir offert l'opportunité d'effectuer mon stage au sein de cette institution.

- Je tiens à remercier M. MEYE NTYAM Pierre Alex, Directeur des Système Informatiques (DSI), pour son accueil, sa disponibilité et ses précieux conseils.
- Je remercie sincèrement Monsieur NDJENG NKOLO Éric Georges chef Service du Développement Maintenance et Infrastructure (SDMI), mon encadreur professionnel, pour sa disponibilité constante, son écoute attentive, ainsi que ses précieux conseils et orientations tout au long de cette expérience.
- Je souhaite également remercier Madame MAYEUGHE Rehanatou pour son suivi rigoureux, son accompagnement et ses conseils avisés qui ont grandement contribué à la réussite de ce stage.
- Je remercie également Madame BOLO ASSENA Merveille, pour sa disponibilité, ses conseils et ses orientations.
- Je tiens également à remercier Messieurs ; AMOU François, LINDJECK Éric, MPONGO Edmond, ATEBA Yves, ODOGO Alain, TAMBI ENOW, Omar EL FAROUK pour leurs disponibilité, conseils, orientations et collaboration.
- J'exprime toute ma gratitude à l'ensemble des agents du Service Développement, Maintenance et Infrastructure, ainsi qu'à toute l'équipe de la Direction des Systèmes d'Information, pour leur accueil chaleureux, leur collaboration et leur disponibilité au quotidien.
- Enfin, je remercie mes camarades de stage avec qui j'ai partagé cette expérience enrichissante, ainsi que ma famille dont le soutien indéfectible m'a toujours accompagné et encouragé.

SOMMAIRE

REMERCIEMENTS	II
SOMMAIRE	III
LISTE DES TABLEAUX	IV
LISTES DES FIGURES	V
INTRODUCTION GÉNÉRALE	1
PARTIE 1 : PRÉSENTATION DE LA CNPS ET ACCUEIL	3
INTRODUCTION	4
ABSTRACT	5
ACCUEIL ET INSERTION	6
PRESENTATION DES TRAVAUX	13
PHASE 1 : REALISATION D'UNE APPLICATION WEB DE CREATION DE COMPTE	18
DOSSIER D'ANALYSE	19
I. CAPTURES DES INTERFACES DES PAGES DE L'APPLICATION.	28
PHASE 2 : TEST DE PRISE EN MAIN DE JENKINS	32
II. INSTALLATION DE JENKINS ET GIT SOUS LE SERVEUR	33
RECOMMANDATIONS	46
JUSTIFICATION DES RECOMMANDATIONS	49
PERSPECTIVES	54
PERSPECTIVE DE RESTRUCTURATION DE L'ARCHITECTURE	55
CONCLUSION GENERALE	60
ABREVIATIONS ET MOT-CLES	61
WEBIOGRAPHIE	62
TABLES DE MATIÈRES	63

Liste des tableaux

Tableau 1: signalétique de la CNPS.....	7
Tableau 2: Etude comparative entre jenkins.war et jenkins.msi.....	35
Tableau 3 : Détermination sur le choix du langage pipeline a prendre	40

Listes des figures

Figure 1: Plan de localisation de la CNPS – source (CNPS)	11
Figure 2: Organigramme de la CNPS (source – S-COM/DECT/CNPSErreur! Bookmark not defined.	
Figure 3: Organigramme de la DSI (source – Photoshop)	12
Figure 4: exemple de contenu de requette POST Request.	21
Figure 5: exemple de preparation d'une requette SQL	21
Figure 6: Réponse de succès	22
Figure 7: Réponse d'échec	22
Figure 8: diagramme de cas d'utilisation du système.	24
Figure 9: Diagramme de classe	25
Figure 10: Diagramme de composants.	26
Figure 11: Diagramme de Déploiement	27
Figure 12: . page d'accueil et de connexion	28
Figure 13: . page d'inscription	28
Figure 14: page de modification de profil	29
Figure 15 : Page dashboard	29
Figure 16: Interface Base de Données.	30
Figure 17: Création d'un dossier, dépôt git	33
Figure 18: Ajout du JDK aux Variables d'environnement	34
Figure 19 : lancement du Jenkins.war	36
Figure 20: Création du compte Jenkins	36
Figure 21: Interface de Jenkins	37
Figure 22: Création d'identifiants git	37
Figure 23: Clonage du depot central	37
Figure 24: Ajout d'une fonctionnalité	38
Figure 25: Création d'un job (tache)	38
Figure 26: Configuration de communication entre git et Jenkins	39
Figure 27: Jenkinsfile	41
Figure 28: Détaille du pipeline	42
Figure 29: Architecture Monolithe	51

MISE EN ŒUVRE D'UN PIPELINE D'INTÉGRATION ET DE DÉPLOIEMENT CONTINUE (CI/CD) BASÉ SUR JENKINS ET GIT AU SEIN DE LA CNPS CAS DE SAPELLI EnergiZer

Figure 30. Architecture Monolithe Moderne	52
Figure 31. architecture microservice	53
Figure 32. fichier JSP	55
Figure 33. Frontend séparé de la JSP existante	56
Figure 34. Backend séparé de la JSP existante	56
Figure 35. Base de données.	57
Figure 36. presentation	57
Figure 37. Jenkinsfile	58
Figure 38. Flux du Processus du CICD.	59

INTRODUCTION GÉNÉRALE

Nouvellement sortie de l'école après l'obtention de mon diplôme, et conformément aux exigences de notre parcours académique, nous sommes appelés à effectuer un stage professionnel. Cette étape constitue un levier essentiel pour faciliter notre insertion dans le monde du travail, appliquer les connaissances théoriques acquises en formation, et développer des compétences pratiques en milieu réel.

À notre arrivée à la **Caisse Nationale de Prévoyance Sociale (CNPS)**, où s'est déroulé notre stage du 13 janvier au 11 juillet 2025, nous avons d'abord observé le fonctionnement général de la cellule de développement informatique. Très rapidement, notre attention s'est portée sur l'application métier "**EnergiZer**", au cœur des processus numériques de la CNPS.

À travers nos différents échanges avec les membres de l'équipe de développement, plusieurs constats ont été relevés :

- Une collaboration parfois désorganisée entre les développeurs ;
- Des conflits fréquents liés à la modification simultanée du code source ;
- Des difficultés à assurer une maintenance efficace de l'application ;
- Une peur d'introduire de nouvelles fonctionnalités, de crainte de provoquer des erreurs ou des redondances dans le système existant.

Face à ces enjeux, la nécessité de mettre en œuvre une solution d'automatisation du développement et du déploiement s'est imposée. C'est ainsi que le thème de mon stage a été défini comme suit:

« Mise en œuvre d'un pipeline CI/CD avec Jenkins et Git au sein de la CNPS pour l'application EnergiZer ».

L'objectif principal de ce projet était de concevoir une chaîne d'intégration et de déploiement continue permettant :

- L'intégration continue du code via un dépôt Git local ;
- Le déclenchement automatique des processus de build, de test et de déploiement grâce à Jenkins ;
- L'amélioration de la qualité du développement logiciel et la réduction des erreurs humaines ;
- La mise en place d'un système de versionning pour optimiser la collaboration entre les

MISE EN ŒUVRE D'UN PIPELINE D'INTÉGRATION ET DE DÉPLOIEMENT CONTINU (CI/CD) BASÉ SUR JENKINS ET GIT AU SEIN DE LA CNPS CAS DE SAPELLI ENERGIZER

équipes et assurer la traçabilité des modifications.

Avant d'implémenter directement cette solution sur l'application *EnergiZer*, il était indispensable de **concevoir une application prototype**. Cette étape préparatoire nous a permis de :

- Mieux comprendre les principes fondamentaux du CI/CD ;
- Expérimenter les outils choisis (Jenkins, Git, etc.) dans un environnement maîtrisé ;
- Tester, simuler et ajuster les différentes étapes du pipeline avant son déploiement réel.

La mise en place de ce prototype a constitué une phase d'apprentissage pratique essentielle, garantissant une meilleure maîtrise des outils et assurant une implémentation plus fiable et sécurisée sur le projet final.

Avant de passer à la configuration proprement dite de Jenkins, une autre étape cruciale a consisté à unifier les fichiers sources de l'application dans un **référentiel Git unique**, afin d'assurer une gestion centralisée, cohérente et suivie du code. Cette démarche visait à faciliter l'intégration continue, la traçabilité des modifications, et une collaboration plus efficace entre les membres de l'équipe.

Ce rapport présentera dans un premier temps la CNPS et ses missions. Ensuite, il détaillera les différentes étapes ayant conduit à la mise en œuvre du pipeline CI/CD : la création du prototype, l'organisation du code dans un dépôt structuré, la configuration de Jenkins, l'automatisation des tâches (build, test, déploiement), ainsi que les résultats et perspectives liés à cette implémentation.

PARTIE 1 : PRÉSENTATION DE LA CNPS ET ACCUEIL

Dans cette première partie, nous allons présenter en détail la Caisse Nationale de Prévoyance Sociale (CNPS), l'institution qui nous a accueillis pour notre stage. Nous commencerons par une description générale de la CNPS, en abordant ses missions, son rôle dans le système de protection sociale camerounais et sa structure organisationnelle. Ensuite, nous décrirons les conditions d'accueil au sein de l'établissement, les formalités administratives effectuées lors de notre intégration, ainsi que l'encadrement fourni par les responsables pendant toute la durée du stage. Cette partie servira à poser les bases pour une meilleure compréhension du cadre dans lequel nos travaux ont été réalisés.

INTRODUCTION

Dans le cadre de notre parcours de formation professionnelle, nous avons eu l'opportunité d'effectuer un stage de six mois au sein de la Caisse Nationale de Prévoyance Sociale (CNPS), une institution publique chargée de la gestion des prestations sociales des travailleurs assurés au Cameroun. Ce stage, qui s'inscrit dans une logique d'insertion professionnelle et de renforcement des compétences pratiques, a débuté le 13 janvier 2025 et s'est déroulé jusqu'au 12 juillet 2025.

L'objectif principal de ce stage était de concevoir et de mettre en œuvre un pipeline CI/CD (Intégration Continue et Déploiement Continu) en utilisant les outils Jenkins et Git, afin d'automatiser le cycle de développement logiciel au sein de la CNPS. Cette mission s'inscrit dans une démarche d'amélioration de la productivité des équipes techniques, de fiabilisation des déploiements applicatifs et de modernisation des processus informatiques internes.

La CNPS, structure à la fois stratégique et sensible, manipule quotidiennement de grandes quantités de données personnelles et financières. La fiabilité, la sécurité et la continuité des services numériques y sont donc des enjeux majeurs. C'est dans ce cadre que nous avons été intégrés à l'équipe des systèmes d'information, avec pour tâche la mise en place progressive d'un pipeline CI/CD, allant de l'unification des fichiers dans un référentiel Git centralisé, jusqu'à la configuration et l'automatisation des étapes clés avec Jenkins.

Dans ce rapport, nous présenterons d'abord l'environnement professionnel de la CNPS et les conditions dans lesquelles nous avons été accueillis. Nous détaillerons ensuite les différentes étapes de réalisation de notre mission, les outils utilisés, les défis rencontrés, ainsi que les compétences pratiques que nous avons pu développer tout au long de cette expérience enrichissante.

ABSTRACT

As part of our professional training program, we had the opportunity to complete a six-month internship at the Caisse Nationale de Prévoyance Sociale (CNPS), a public institution responsible for managing social benefits for insured workers in Cameroon. This internship, aimed at professional integration and strengthening practical skills, took place from January 13 to July 12, 2025. The main objective was to design and implement a CI/CD pipeline (Continuous Integration and Continuous Deployment) using Jenkins and Git to automate the software development lifecycle within CNPS. This mission aimed to improve the productivity of technical teams, ensure reliable application deployments, and modernize internal IT processes. The CNPS, a strategic and sensitive organization, handles large volumes of personal and financial data daily. Ensuring the reliability, security, and continuity of its digital services is therefore a major concern. Within this context, we were integrated into the information systems team to progressively set up a CI/CD pipeline, from unifying files into a centralized Git repository to configuring and automating key steps with Jenkins. This report first presents the professional environment of CNPS and the conditions of our integration. It then details the different stages of our mission, the tools used, the challenges encountered, and the practical skills developed throughout this enriching experience.

ACCUEIL ET INSERTION

Accueil au sein de la structure

Le lundi 13 janvier 2025 à 07h30 a marqué le début effectif de mon stage à la CNPS. Dès mon arrivée, après un accueil chaleureux au secrétariat, j'ai été directement orientée vers la Direction des Systèmes d'Information (DSI). Lors d'un premier entretien, les objectifs et l'organisation générale du stage m'ont été présentés. J'ai ensuite été introduite aux différents membres de l'équipe de la DSI, ce qui m'a permis de mieux comprendre le fonctionnement du service.

Par la suite, j'ai été conduite à l'atelier de maintenance, où j'ai rencontré mes collègues stagiaires ainsi que les responsables en charge de cet espace technique. Cette immersion initiale a été l'occasion de découvrir l'environnement professionnel dans lequel j'allais évoluer et d'appréhender les missions qui me seraient confiées tout au long de mon stage.

Présentation General de la CNPS

Fiche Signalétique de la structure

Tableau 1: signalétique de la CNPS

Raison sociale	Caisse Nationale de Prévoyance Sociale
Sigle	CNPS
Logo	
Régime	Général
Création	1967
Directeur Général	Noël Alain Olivier MEKULU MVONDO AKAME
Siège	Yaoundé
Activité	Sécurité Sociale
Téléphone	22 23 08 31
Fax	22 22 57 55
E-mail	www.CNPS.com
Adresse	441 Ydé

Historique

Le système Camerounais de protection sociale a connu deux phases d'évolution : celle de **l'avant indépendance** et celle de **l'après indépendance**.

La Protection Sociale Avant l'Indépendance

La substance du système social camerounais se décline de manière par l'ordonnance du 06 décembre 1945 mise en place par la France, créant une caisse de compensation des prestations familiales dont le siège est à Douala. Il importe toutefois de souligner qu'avant cet ordonnancement, quelques textes coloniaux posaient déjà les jalons d'un embryon de sécurité sociale au Cameroun; pour l'illustrer, il y'a lieu de citer le décret du 17 novembre 1937 qui, pour la première fois, posait le principe de l'indemnisation directe, par les employeurs, des accidents du travail survenus à leur travailleurs; on citera aussi le décret du 7 janvier 1944 portant réglementation du travail indigène du Cameroun, notamment dans son titre VI qui instituait un système de réparation des accidents de travail et des maladies professionnelles.

Le code du travail des pays d'outre-mer promulgué le 15 décembre 1952 viendra mettre un certain ordre dans les nombreuses initiatives des divers textes de l'administration coloniale tendant à mettre en place au Cameroun quelques germes des assurances sociales. Le bénéfice de la caisse de compensation est étendu à tous les travailleurs (articles 237 du code).

Un décret du 1er juillet 1956 traduit dans les faits l'extension du champ de compétence personnel de la caisse de compensation des prestations familiales.

A la veille des indépendances, la loi n°59/25 du 11 avril 1959 abroge le décret de 1956 et réorganise la caisse. Les prestations familiales comprennent :

- ❖ **Les associations familiales distribuées au titre des enfants mineurs à charge ;**
- ❖ **Les allocations prénatales versées pendant la grossesse ;**
- ❖ **Le salaire unique ou l'allocation de foyer (complément de salaire pour le travailleur dont le conjoint n'a pas d'emploi salarié) ;**
- ❖ **Des prestations en nature.**

Le code du travail de 1952 prévoit également l'extension à tous les travailleurs de la couverture

des risques professionnels (maladies et accidents). Le décret du 24 février 1957 organise la formule. Celle-ci est basée sur l'idée de la responsabilité de l'employeur pour les maladies professionnelles et accidents de travail. Mais les prestations sont servies par un organisme assureur à qui les employeurs versent des cotisations et code de travail de 1952 permet enfin l'extension aux camerounais de la risque vieillesse.

La Protection Sociale Après l'Indépendance

Avec l'indépendance et la réunification, le Cameroun, membre de l'organisation du travail depuis 1960, est amené à adapter sa législation aux normes internationales. Cet ajustement du cadre juridique de la protection sociale s'est fait à travers :

- ❖ La loi **n°67/LF/07 du 12 juin 1967** instituant un code des Prestations Familiales (allocations prénatales, allocations familiales, allocations maternité, indemnités journalières de congé de maternité) ;
- ❖ La loi **n°67/LF/08 du 12 juin 1967** qui crée la **Caisse Nationale de Prévoyance Sociale** en charge de la gestion des prestations familiales ;
- ❖ La loi **n°69/LF/18 du 10 novembre 1969** instituant un régime d'assurance pensions de vieillesse, d'invalidité et de décès. Dans ce régime, le financement est assuré à travers les cotisations sociales recouvrées tant auprès des employeurs que des travailleurs ;
- ❖ L'ordonnance **n°73/17 du 22 mai 1973** portant organisation de la prévoyance sociale qui confie à la Caisse Nationale de Prévoyance Sociale dans le cadre de la politique générale du Gouvernement, le service des diverses prestations prévues par la législation de la protection sociale.
- ❖ La loi **n°77/11 du 13 juillet 1977** portant réparation et prévention des accidents du travail et des maladies professionnelles, qui confie à la Caisse Nationale de Prévoyance Sociale, la couverture et la gestion des risques professionnels, abrogeant ainsi une législation antérieure issue de l'Ordonnance 59/100 du 31 décembre 1959 qui avait confié la gestion de ces risques aux compagnies privées d'assurance.

Mission

La Caisse Nationale de Prévoyance Sociale a pour mission d'assurer dans le cadre de la politique générale du gouvernement, le service de diverses prestations prévues par la législation de protection sociale et familiale. A ce titre, elle couvre trois branches de sécurité sociale dont :

- ❖ **Les Prestations Familiales (PF)**
- ❖ **Les Pensions Vieillesse, d'Invalidité et de Décès (PVID)**
- ❖ **Les Risques Professionnels (RP)**

Le système de sécurité sociale Camerounais est basé sur la solidarité ; ceux qui travaillent cotisent pour ceux admis à faire valoir leurs droits à la retraite. De même, il convient de rappeler que la principale source de financement de la CNPS reste les cotisations sociales que les employeurs et les assurés volontaires réservent mensuellement.

Par ailleurs, dans un environnement socio-économique incertain, ou l'organisme doit assurer ses obligations vis à vis des générations futures, l'astuce a consisté à entreprendre des investissements économiquement et financièrement porteurs. L'immobilier est l'option choisie d'où l'acquisition en janvier 2010 de l'immeuble Camair à Paris, la réhabilitation de l'immeuble ministériel n°1 à Yaoundé, etc., qui

Participent de cette politique d'investissement orientée vers les secteurs productifs et présentant les garanties de sécurité et de pérennité financière.

L'organisme investit également dans l'amélioration constante de l'offre des soins de santé, dans le cadre de son action sanitaire et sociale. C'est ainsi que la rénovation profonde des plateaux technique des structures hospitalières a été, par l'acquisition d'équipement à la pointe des nouvelles technologies.

La CNPS assure, depuis le 03 novembre 2014, la protection sociale des travailleurs indépendants, qui peuvent s'immatriculer dans le cadre de l'assurance volontaire. Ce processus d'extension de la couverture sociale permet de s'immatriculer dans l'un des 42 **centres de prévoyance sociale** de l'organisme à titre volontaire dès l'âge de 14 ans, sous réserve de l'accord parental évidemment.

A la CNPS, le souci est de soutenir le présent, tout en confrontant l'avenir des assurés sociaux, nos véritables patrons

Localisation

Le siège de la CNPS est situé à la place de l'indépendance, à côté de l'hôtel de ville, à Yaoundé Centre, Cameroun.

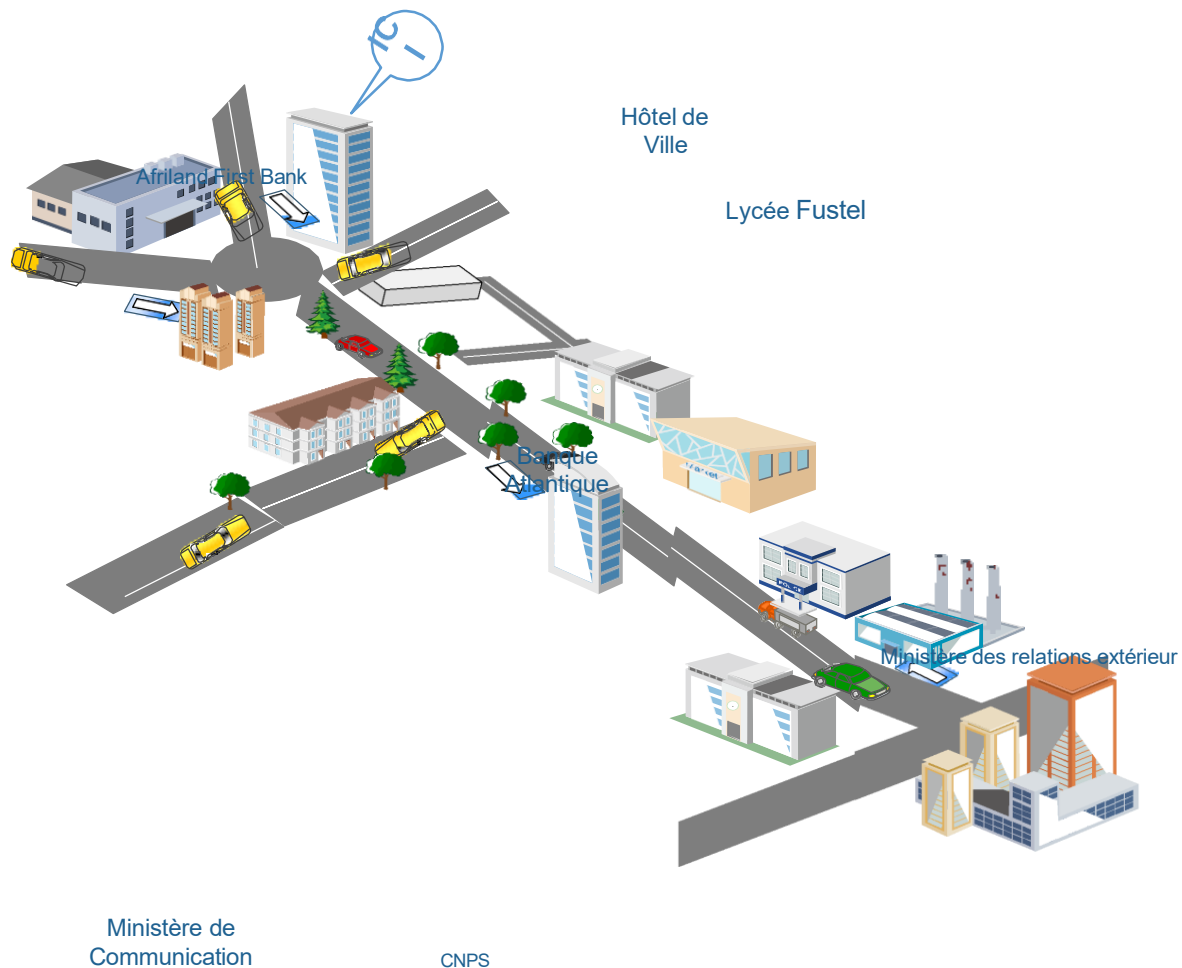


Figure 1: Plan de localisation de la CNPS – source (CNPS)

La **DSI (Direction des Systèmes d'Information)** est la structure de la CNPS qui est chargé d'administrer et de gérer tous les services informatiques (maintenance, réseau, développement, archivage) nécessaire au fonctionnement optimal et efficace des différentes activités de l'entreprise. Dans cette optique, cette direction est composée d'un secrétariat et trois (03) services, à savoir : **Le Service de Développement, Maintenance et Infrastructure(SDMI), le Service de la Sécurité des Systèmes d'Information (SSI), et le Service de la Documentation ou Archives (SD/SA).**

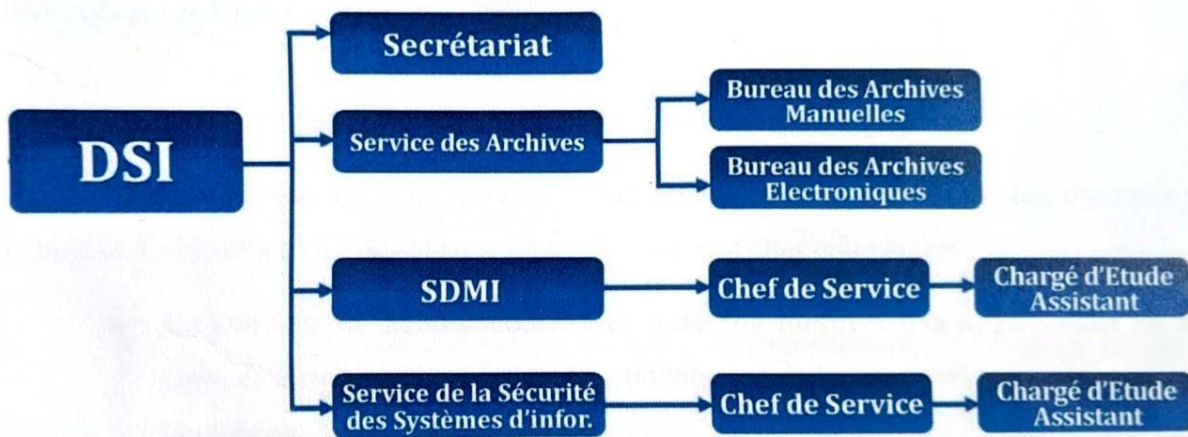


Figure 2: Organigramme de la DSI (source – Photoshop)

PRESENTATION DES TRAVAUX

Durant notre stage, nous avons réalisé plusieurs travaux, dont un projet principal accompagné de tâches secondaires. Notre travail principal consistait à développer une application web avec le framework Quasar pour le frontend et PHP pour le backend. Ensuite, en deuxième partie, nous avons implémenté une approche CI/CD avec Jenkins et une instance locale de Git.

Avant de mettre en place le CI/CD, il était essentiel de répondre aux questions suivantes :

- Qu'est-ce que le CI/CD ?
 - Pourquoi l'utiliser ?
 - Quels sont ses avantages et ses inconvénients ?
 - Comment l'implémenter ?
 - Quelle approche adopter pour sa mise en place ?

Contexte et problématique

Si nous revenons aux anciennes méthodes de développement traditionnelles, nous constatons que les équipes de développement rencontraient de nombreuses difficultés. En effet, il était compliqué d'intégrer de nouvelles fonctionnalités dans une application principale sans engendrer des régressions dans le code. Les erreurs n'étaient souvent détectées qu'après la mise en production, rendant leur correction plus complexe et pouvant provoquer des interruptions de service. De plus, le déploiement manuel était une tâche chronophage, ce qui rendait difficile la mise en production d'applications robustes, notamment pour des projets complexes. Ces contraintes entraînaient également un manque de synergie au sein des équipes de développement, ralentissant ainsi le processus global. Face à ces défis, un nouveau processus est apparu pour y remédier : le CI/CD.

Le CI/CD : Définition et avantages

Le CI/CD, qui signifie **Intégration Continue (CI)** et **Déploiement/Distribution Continue (CD)**, est un processus qui optimise les étapes du développement logiciel. Il permet d'automatiser la **construction (compilation)** des fichiers de code en builds (optimisation, minimisation du code, édition des liens), les **tests**, ainsi que le **déploiement en production**.

Il repose sur trois composants clés :

1. **Les développeurs** : ils écrivent et modifient le code.
2. **Le système de gestion de versions (Git)** : il permet de récupérer, stocker et gérer les différentes versions du code.
3. **L'outil CI/CD (comme Jenkins)** : il récupère le code, effectue les builds, exécute les tests et procède au déploiement automatique en production.

Avantages du CI/CD

Grâce au CI/CD, plusieurs bénéfices sont observés :

- **Gain de temps** : grâce à l'automatisation des tâches répétitives.
- **Robustesse accrue** : les erreurs sont détectées plus rapidement, dès les premières étapes.
- **Meilleure collaboration** : le processus favorise une synergie efficace entre les membres de l'équipe.
- **Réduction des régressions** : les tests fréquents assurent une meilleure stabilité du code et rendent les mises en production plus fiables.

Ainsi, le CI/CD s'impose aujourd'hui comme une **bonne pratique essentielle** dans le développement logiciel, car il corrige efficacement les limites des méthodes traditionnelles et améliore la qualité globale des applications.

Approches d'implémentation du CI/CD

Il existe plusieurs manières d'implémenter le CI/CD, selon l'environnement de développement :

- **CI/CD avec un outil tiers (Jenkins)** et une instance locale de Git.
- **CI/CD avec GitHub**, qui intègre son propre outil CI/CD (*GitHub Actions*) ou peut être couplé avec Jenkins.
- **CI/CD avec GitLab**, qui dispose d'un système CI/CD intégré.
- **CI/CD avec Jenkins seul**, utilisé indépendamment pour orchestrer les processus d'intégration et de déploiement continu.

Inconvénients du CI/CD

Malgré ses nombreux avantages, le CI/CD présente aussi des limites qu'il convient de prendre en compte :

- **Complexité initiale élevée**
La mise en place d'un pipeline CI/CD demande du temps, des outils spécifiques et une architecture logicielle bien pensée.
- **Besoin de compétences spécialisées**
Toute l'équipe doit comprendre les processus CI/CD, les outils utilisés (Jenkins, Git, GitHub, GitLab CI...), et être capable de maintenir les scripts d'automatisation.
- **Coûts d'infrastructure**
Les serveurs d'intégration, les systèmes de stockage, et les ressources nécessaires peuvent engendrer des coûts importants, surtout pour les petites structures.
- **Risque de déploiement de bugs**
Si les tests automatisés sont incomplets ou mal conçus, le processus peut envoyer en production des erreurs non détectées.
- **Maintenance continue du pipeline**
La pipeline CI/CD devient un élément à part entière du projet qu'il faut faire évoluer, corriger et adapter en permanence (mise à jour des outils, refactorisation des scripts, etc.).

- Étude comparative sur les différentes approches

APPROCHES	AVANTAGES	INCONVENIENTS
CICD +OUTIL TIER + GIT LOCAL	Offre un contrôle total et une sécurité optimale, car tout est effectué en local sur notre propre serveur.	- Nécessite une configuration manuelle plus complexe.
CICD + OUTIL TIER+GITHUB	Configuration automatique simplifiée.	- Pas de contrôle total sur l'infrastructure. - Sécurité dépendante du cloud. - Nécessite un abonnement pour certaines fonctionnalités avancées.
CICD +GITLAB	- Intègre un système CI/CD natif sans dépendance à un outil tiers. - Implémentation rapide et fluide.	-Exige une bonne maîtrise des configurations. - Hébergement sur un serveur cloud, impliquant des préoccupations de sécurité et un manque de contrôle total. - Peut nécessiter un abonnement payant
CICD + JENKINS	- Indépendance vis-à-vis d'un service de gestion de versions (SVC). - Accessible aux non-développeurs (pas besoin de maîtriser Git). - Solution flexible et personnalisable.	- Collaboration en équipe plus difficile. - Risque d'erreurs humaines (manipulation manuelle des fichiers). - Absence d'historique des modifications, rendant le retour à une version précédente complexe.

Tableau : étude comparative des différentes approches du CI/CD

Pourquoi utiliser Jenkins ?

- **Automatise le CI/CD** → Build, test et déploiement sans intervention manuelle.
- **Open-source et gratuit** → Aucune licence requise.
- **Flexible** → Compatible avec tous les langages et outils (Git, Docker, Kubernetes, etc.).
- **Personnalisable** → Plus de **1 800 plugins** disponibles.
- **Indépendant** → Fonctionne sur **Windows, Linux, macOS** et peut être installé en **local ou cloud**.

Pourquoi choisir Jenkins comme outil tier avec une instance locale de git ?

- ✓ **Pour un contrôler entièrement ton code** sans dépendre du cloud.
- ✓ **Pour travailler dans un environnement sécurisé** (exemple ; entreprise)
- ✓ **Pour une solution performante et rapide** sans dépendre d'Internet.

Détail du Projet

Durant notre stage, nous avons été assigné la mission d'implémenter un pipeline CICD en entreprise et pour réaliser cette mission, il était question donc pour nous de faire un prototype d'implémentation d'un pipeline CICD avec Jenkins et c'est ainsi que nous avons en **première partie de réaliser une mini application web qu'en phase deux, sera implémenter vers un pipeline CICD**. La mini application était une application de création de comptes et pour réaliser cette application nous avons utilisé des outils de développement tel que ; **Vue.js avec le Framework Quasar pour la vue utilisateur (Frontend) et pour le Backend nous avons utilisé PHP, MYSQL pour la Base de Données et le serveur de développement Laragon et Apache pour le déploiement de notre application web.**

PHASE 1 : REALISATION D'UNE APPLICATION WEB DE CREATION DE COMPTE.

- Technologies utilisées ; Vue.js, Node.js le Framework Frontend Quasar, PHP, le serveur de développement Laragon, MYSQL pour la base de données.
- Objective de l'application ; L'application web doit permettre à un utilisateur de créer un compte par l'inscription et avoir accès à ce compte par la connexion et un administrateur ayant possibilité d'ajouter, supprimer un utilisateur.

PHASE 2 : IMPLEMENTATON D'UN CICD AVEC GIT ET JENKINS Cas Pratique (EN LOCAL)

PHASE 1 : REALISATION D'UNE APPLICATION WEB DE CREATION DE COMPTE

La réalisation de cette application aura pour but de faire les premiers tests de prise en main du CICD et la compréhension du concept. La mini application a pour fonctionnalités de créer des comptes et avoir accès à un Dashboard ayant la possibilité de modifier leurs profils.

Prérequis :

Technologies utilisées ;

- En Frontend nous avons utilisé les Frameworks JavaScript ; Vue.js, Node.js et Quasar.
- En Backend PHP.
- MSQL pour la base de données.
- Comme serveur d'hébergement Laragon.

DOSSIER D'ANALYSE

Le dossier d'analyse est un document crucial qui guide la sélection de la méthode d'analyse appropriée à utiliser tout au long du projet. Il sert non seulement à expliciter les différentes facettes du projet à mettre en œuvre mais aussi à baliser le chemin pour une exécution sans faille. En d'autres termes, il jette les bases solides nécessaires pour une mise en œuvre réussie et systématique du projet.

Ce document d'analyse décrit l'architecture logicielle et les modèles UML d'une application web de création de comptes utilisateurs. Le système permet aux utilisateurs de s'enregistrer, se connecter, et aux administrateurs de gérer les comptes. Le projet utilise Quasar (Vue.js) pour le frontend, PHP pour le backend, et MySQL comme base de données.

1. Présentation de l'Architecture de l'application

1.1 Architecture logique (3-tiers)

Notre application suit une architecture 3-tiers de trois couches ;

- **Frontend (Vue.js - Quasar)** : gestion de l'interface utilisateur.
- **Backend (PHP)** : logique métier, API REST, validation, enregistrement.
- **Base de données (MySQL)** : persistance des données utilisateurs.

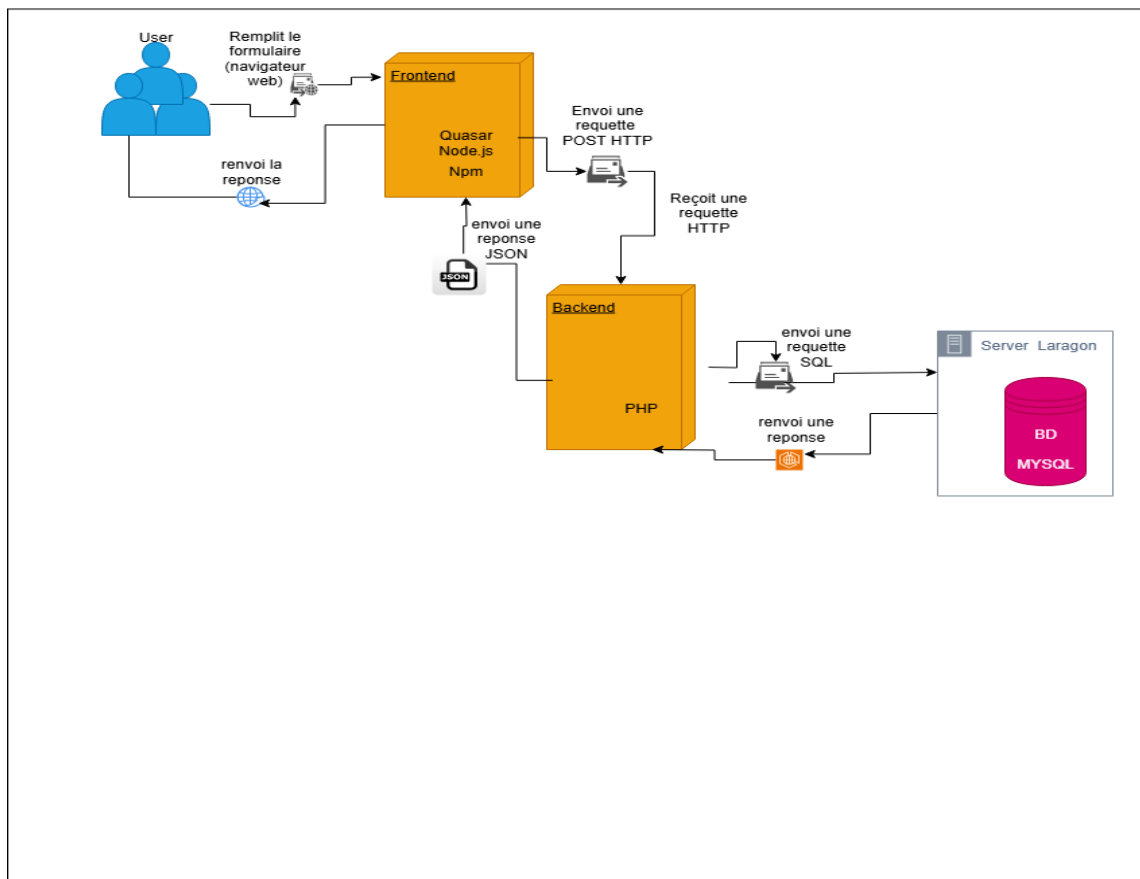


Fig. Architecture 3-couches

1.2 Description de l'architecture logique

L'architecture logique de l'application repose sur un enchaînement clair entre le **frontend** (Quasar/Vue.js), le **backend** (PHP), et la **base de données** (MySQL). Ce modèle suit le principe d'une **architecture en trois couches** : présentation, logique métier, et persistance des données.

Voici une description détaillée du **cycle de traitement d'une requête**, illustrant le fonctionnement typique lors de la création d'un compte utilisateur :

1. Interaction utilisateur (couche présentation)

L'utilisateur saisit ses informations (nom, email, mot de passe) dans un formulaire à l'aide de l'interface développée avec **Quasar (Vue.js)**. Lorsqu'il clique sur « S'inscrire », une requête est déclenchée.

2. Envoi de la requête HTTP (communication client-serveur)

Le frontend envoie une requête HTTP de type **POST** vers l'API PHP
(http://localhost/api/register.php). Cette requête contient les données saisies sous forme de
JSON, par exemple :

```
json
{
  "nom": "Agathe",
  "email": "agathe@example.com",
  "mot_de_passe": "123456"
}
```

Figure 3: exemple de contenu de requête POST Request.

3. Traitement côté backend (couche logique métier)

Le backend développé en PHP :

- Reçoit et **interprète les données JSON** envoyées.
- Effectue des **validations** (vérification d'un email existant, sécurité du mot de passe...).
- Prépare une **requête SQL d'insertion** vers la base de données :

```
sql
INSERT INTO utilisateurs (nom, email, mot_de_passe)
VALUES ('Agathe', 'agathe@example.com', 'hash_du_mdp');
```

Figure 4: exemple de preparation d'une requête SQL

4. Exécution de la requête par la base de données (couche données)

Le **SGBD MySQL** exécute la requête :

- Si l'opération réussit, il renvoie un **succès** (vrai, identifiant inséré...).
- Si une erreur survient (ex. : email déjà existant), une **erreur SQL** est renvoyée.

5. Réponse du backend au frontend

Le backend **analyse la réponse** de la base de données et génère une réponse JSON appropriée :

- En cas de succès :

```
json
{
  "success": true,
  "message": "Compte créé avec succès."
}
```

Figure 5: Réponse de succès

- En cas d'échec :

```
json
{
  "success": false,
  "message": "Cet email existe déjà."
}
```

Figure 6: Réponse d'échec

6. Retour vers l'interface utilisateur

Le frontend **interprète la réponse** JSON reçue, puis affiche un message à l'utilisateur :

- *"Bienvenue Agathe ! Votre compte a été créé avec succès."*
- *"Erreur : cet email est déjà utilisé."*

2. Présentation des diagrammes

Afin de mieux représenter la structure fonctionnelle et technique de l'application de création de

comptes, plusieurs diagrammes UML ont été élaborés. Ces diagrammes permettent de visualiser les interactions entre les différents éléments du système, tant du point de vue des utilisateurs que de l'architecture logicielle.

Les modèles UML suivants sont présentés dans cette section :

- **Diagramme de cas d'utilisation** : il décrit les principales interactions entre les acteurs (utilisateur et administrateur) et les fonctionnalités offertes par le système.
- **Diagramme de classes** : il modélise les entités principales manipulées dans l'application ainsi que leurs attributs et méthodes.
- **Diagramme de composants** : il illustre l'organisation logicielle de l'application et les relations entre les différents modules.
- **Diagramme de déploiement** : il montre la répartition physique des composants de l'application sur les différents environnements et serveurs utilisés.

Ces représentations UML jouent un rôle essentiel dans la compréhension, la documentation et la maintenance du système, en offrant une vue globale cohérente de l'application.

2.1 Diagramme de cas d'utilisation (usecase)

Ce diagramme montre les interactions entre les **acteurs** (utilisateur et administrateur) et le **système**.

Cas d'utilisation principaux :

- Créer un compte
- Se connecter
- Modifier son profil
- Supprimer son compte
- (Administrateur) Voir la liste des comptes
- (Administrateur) Activer/Désactiver un compte
- (Administrateur) Supprimer un compte

```

    usecaseDiagram
        actor Users
        actor User
        actor Admin

        usecase UC1[S'inscrire]
        usecase UC2[Créer un compte]
        usecase UC3[Modifier Profil]
        usecase UC4[Modifier mot de passe]
        usecase UC5[Consulter comptes]
        usecase UC6[Gérer Comptes]
        usecase UC7[supprimer]
        usecase UC8[Activer]
        usecase UC9[s'authentifier]

        Users -- UC1
        User -- UC4
        User -- UC5
        Admin -- UC6
        Admin -- UC7
        Admin -- UC8
        Admin -- UC9

        UC1 -.-> UC2 : <<Include>>
        UC1 -.-> UC3 : <<Include>>
        UC6 -.-> UC7 : <<Include>>
        UC6 -.-> UC8 : <<Include>>
    
```

Diagramme de classes

Classes clés :

- 24

- Rôle

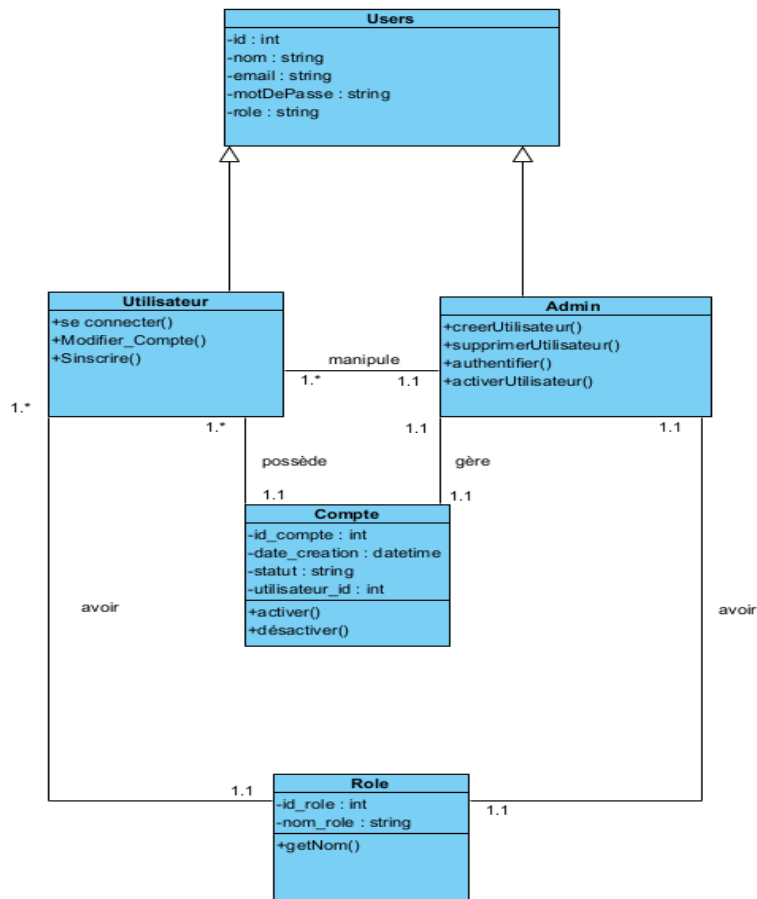


Figure 8:. Diagramme de classe

Ce diagramme montre l'organisation logique des composants logiciels de l'application et leurs interactions. Il est centré sur la communication entre le **frontend Quasar**, le **backend PHP**, et **MySQL**.

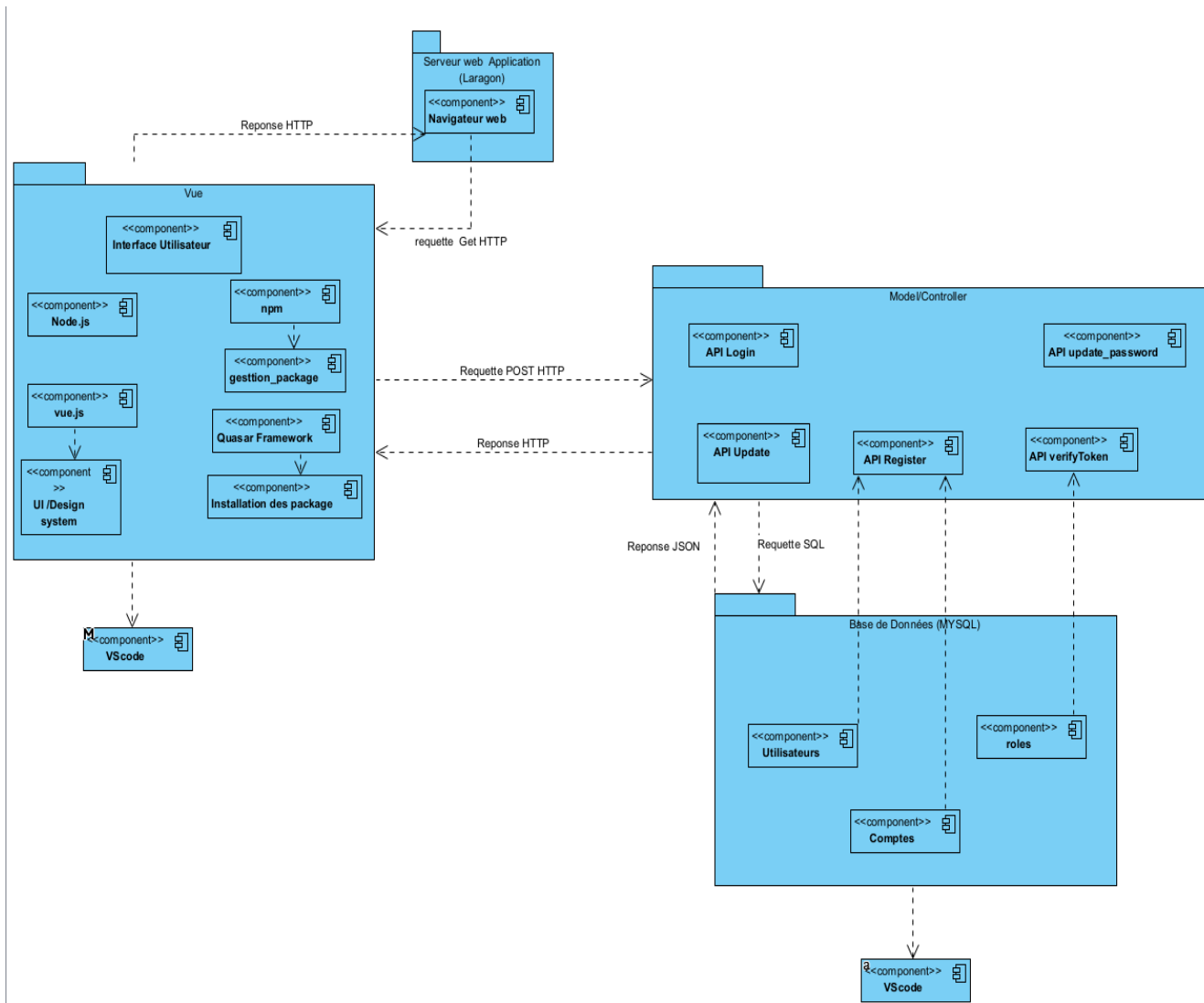


Figure 9:. Diagramme de composants.

2.3 Diagramme de Déploiement

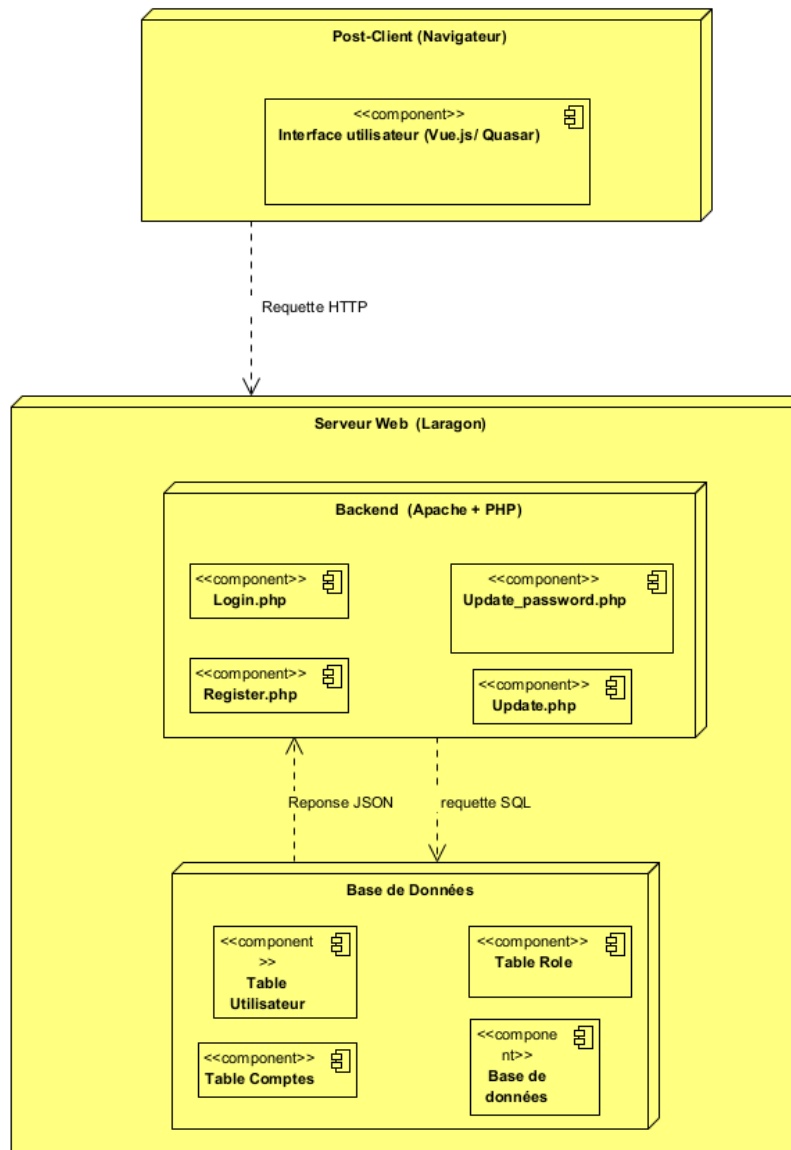


Figure 10:. Diagramme de Déploiement

I. Captures des Interfaces des Pages de l'application.

1. Page d'accueil et de connexion

The screenshot shows the 'Accountify' login page. At the top, there's a blue header with the 'Accountify' logo. Below it, a large blue button labeled 'CONNEXION' with a key icon is on the left, and a blue button labeled 'INSCRIPTION' with a person icon is on the right. Below these buttons are two input fields: 'Adresse Email' with the value 'rose@gmail.com' and 'Mot de passe' with four dots. At the bottom, there's a large blue button labeled 'SE CONNECTER'.

Figure 11: . page d'accueil et de connexion

2. Page d'inscription

The screenshot shows the 'Accountify' registration page. At the top, there's a blue header with the 'Accountify' logo. Below it, a large blue button labeled 'CONNEXION' with a key icon is on the left, and a blue button labeled 'INSCRIPTION' with a person icon is on the right. Below these buttons are four input fields: 'Nom complet', 'Téléphone' (with a red border and a red exclamation mark icon), 'Adresse Email' with the value 'rose@gmail.com', and 'Mot de passe' with four dots. At the bottom, there's a large green button labeled 'S'INSCRIRE'.

Figure 12:.. page d'inscription

3. Page de modification de profil

Accountify

Modifier mon profil

 [CHANGER LA PHOTO](#)

Nom
John Doe

[ENREGISTRER](#)

Figure 13: page de modification de profil

4. Page Dashboard

Tableau de bord Admin

PROFIL DÉCONNEXION

Bienvenue, administrateur !

Liste des utilisateurs

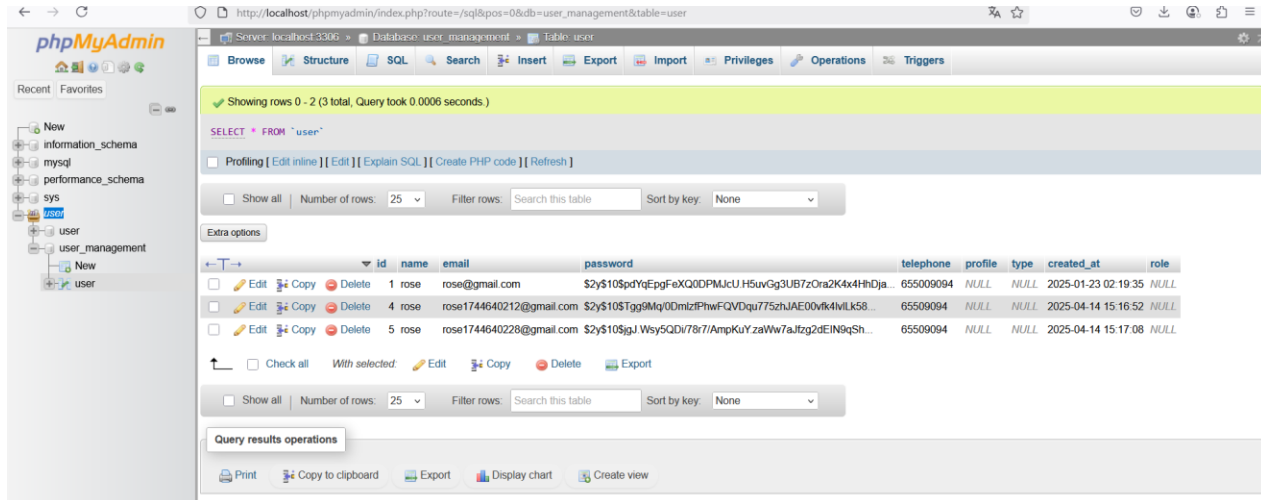
ID	Nom	Email	Date d'inscription	Actions
No data available				

Actions disponibles

[AJOUTER UN UTILISATEUR](#) [SUPPRIMER UN UTILISATEUR](#)

Figure 14 : Page dashboard

5. Interface Base de Données



The screenshot shows the phpMyAdmin interface for a MySQL database named 'user_management'. The 'user' table is selected, and its structure is displayed. The table has columns: id, name, email, password, telephone, profile, type, created_at, and role. There are 3 rows of data.

	id	name	email	password	telephone	profile	type	created_at	role
☐	1	rose	rose@gmail.com	\$2y\$10\$pdYqEpgF-eXQ00PMjCU H5uvGg3UB7zOra2K4x4HhDja...	65509094	NULL	NULL	2025-01-23 02:19:35	NULL
☐	4	rose	rose1744640212@gmail.com	\$2y\$10\$Tgg9Mq/0dmzIFhwFQVDQu775zhJAE00vfk4ivLK58...	65509094	NULL	NULL	2025-04-14 15:16:52	NULL
☐	5	rose	rose1744640228@gmail.com	\$2y\$10\$gJl Wsy5QDI78r7/AmpKuYzaiWw7aJtZg2dEINqSh...	65509094	NULL	NULL	2025-04-14 15:17:08	NULL

Figure 15. Interface Base de Données.

Bien que notre application fonctionne normalement, elle présente certaines contraintes :

- Si nous souhaitons agrandir notre application en travaillant avec une équipe de développement, il ne sera pas évident de maintenir le versionnage de l'application.
- En cas d'ajout d'une nouvelle fonctionnalité, notre application pourrait être affectée (interruption de service, ou encore un simple ajout ou une modification du code pouvant représenter un risque, surtout en production).
- La maintenance ou même une mise à jour des technologies utilisées peuvent également affecter l'application.
- Le travail en équipe peut engendrer des conflits dans le code de notre application, ce qui nuit à son évolutivité.

Ces risques peuvent donc freiner l'évolution de notre application. La question est de savoir comment — et par quels moyens — nous allons résoudre ces problèmes afin d'assurer une intégration continue des nouvelles fonctionnalités, un déploiement continu permettant la maintenance de notre application même lors de modifications, ainsi qu'une solution garantissant le versionnage et réduisant les risques de conflits dans le code, tout en assurant une traçabilité dans le cycle de développement. Il s'agit également de gagner du temps dans le développement en automatisant les tâches telles que les tests, les builds et le déploiement.

C'est ainsi que nous avons choisi comme solution l'implémentation d'un pipeline CI/CD, suivant les bonnes pratiques de développement, qui assurera l'intégration continue, le déploiement continu de notre application, la gestion du versionnage du code, tout en garantissant une traçabilité et en réduisant les conflits ainsi que les risques de pertes de fichiers.

Dans cette partie 2, nous allons donc implémenter un pipeline CI/CD.

PHASE 2 : TEST DE PRISE EN MAIN DE JENKINS

Afin de procéder à l'implémentation du pipeline CI/CD, il est essentiel de commencer par une phase de prise en main du concept. Pour mieux en comprendre les mécanismes et les enjeux, nous avons choisi d'utiliser notre propre application comme environnement de test. Ainsi, après avoir finalisé le développement de l'application, nous passerons à la phase de test d'intégration continue et de déploiement continu à l'aide des outils Jenkins et Git.

Prérequis :

- ✓ Installer Jenkins sous un serveur
- ✓ Installer Git sous le serveur
- ✓ Installer le JDK (version 17-21)
- ✓ Créer un dépôt central (dépôt bare)
- ✓ Faire communiquer Git et Jenkins lors de la configuration de Jenkins.

II. INSTALLATION DE JENKINS ET GIT SOUS LE SERVEUR

1. INSTALLATION DE GIT

Pour installer Git, nous allons nous rendre dans le site officiel et installer **Git** via le site :

<https://git-scm.com/>

- Créer un dossier et un dépôt git en local via la commande suivant :

```
C:\Users\sarah.mangia>where git
C:\Program Files\Git\cmd\git.exe

C:\Users\sarah.mangia>mkdir repos

C:\Users\sarah.mangia>cd repos

C:\Users\sarah.mangia\repos>git init --bare repos
Initialized empty Git repository in C:/Users/sarah.mangia/repos/repos/

C:\Users\sarah.mangia\repos>
```

Figure 16: Création d'un dossier, dépôt git

Cette image nous avant d'abord essayé de trouver l'emplacement où de git, puis nous avons créé un dossier avec la commande **mkdir nom_dossier (ex : repos)**. Ensuite nous devons nous positionner dans l'emplacement du dossier avec la commande **cd nom_dossier** ensuite nous allons créer un dépôt central avec la commande **git init --bare nom_depot**.

2. INSTALLATION DE JENKINS

Pour installer Jenkins il faudrait d'abord installer le JDK car il est basé en java. Avant d'installer le JDK, il est important de prendre en considération la compatibilité du JDK avec JENKINS. Pour notre part nous avons pris la version 17 du JDK qui est compatible a la version 2.492.2. Mais la version 21 du JDK est aussi compatible a la version 2.492.2 de Jenkins.

a. Installation du JDK

Nous allons nous rendre dans le site officiel ou nous allons installer le JDK

<https://www.oracle.com/java/technologies/downloads/>

Une fois le JDK installé, nous allons copier son emplacement et nous rendre dans les variables d'environnement ou nous allons ajouter l'adresses du JDK.

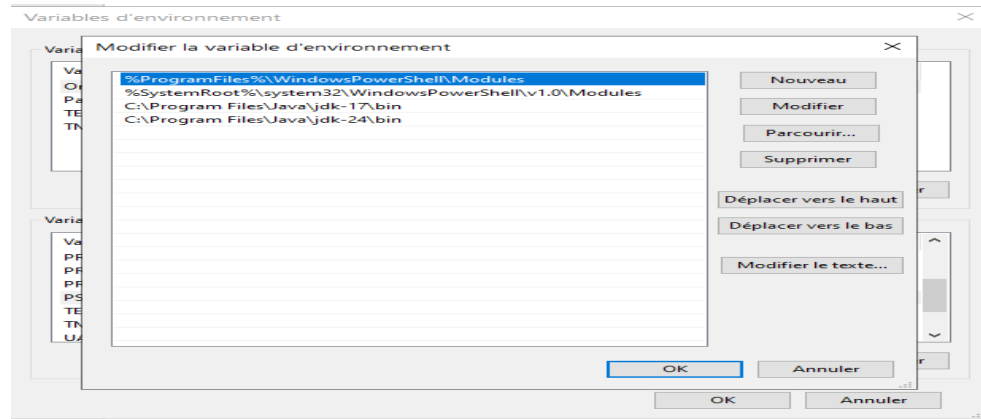


Figure 17: Ajout du JDK aux Variables d'environnement

b. Installation de Jenkins

Une fois le JDK installé, nous allons nous rendre sous le site officiel de Jenkins via le site <https://www.jenkins.io/> et nous allons télécharger le **fichier .war de jenkins**. Nous pouvons aussi installer Jenkins.msi.

c. Justificatif du choix du fichier .war de jenkins et jenkins.msi

Critères	Jenkins .war	Jenkins .msi
Système d'exploitation	Windows, Linux, macOS	Windows uniquement
Installation	Manuelle (via <code>java -jar jenkins.war</code>)	Automatique via un assistant d'installation
Portabilité	Très portable, peut être exécuté sur différents systèmes	Non portable, lié à Windows
Mise à jour	Simple (remplacement du fichier .war)	Nécessite une réinstallation
Démarrage	Doit être lancé manuellement (sauf configuration comme service)	Se lance automatiquement avec Windows

Gestion des services	Nécessite une configuration pour tourner en tant que service	Installe et configure automatiquement Jenkins en tant que service Windows
Intégration avec Tomcat	Oui, peut être déployé sur un serveur d'applications	Non, fonctionne comme une application autonome
Facilité d'utilisation	Plus technique, nécessite des commandes	Simple et accessible via une interface graphique

Tableau 2: Etude comparative entre jenkins.war et jenkins.msi

Choix recommandé : Si nous voulons un contrôle absolu et d'une flexibilité alors nous allons utiliser le Jenkins.war. Pour le démarrer, allons-nous positionner vers son emplacement qui se trouve dans les téléchargements. Nous allons d'abord nous rassurer que le JDK est bien présent avec la commande **java -version** ensuite nous allons lancer jenkins avec la commande ; **java -jar jenkins.War**. Une fois jenkins lancer avec l'url **localhost : 8080 par défaut**, nous serons appelées à remplir le mot de passe par défaut du compte Admin de jenkins et ce code se trouve dans le dossier. Jenkins et nous allons nous rendre dans secrets puis Admin password et une fois cela fait nous allons maintenant confirmer la création du compte :

MISE EN ŒUVRE D'UN PIPELINE D'INTÉGRATION ET DE DÉPLOIEMENT CONTINU (CI/CD) BASÉ SUR JENKINS ET GIT AU SEIN DE LA CNPS CAS DE SAPELLI ENERGIZER

```
C:\Windows\System32\cmd.exe - java -jar jenkins.war
Microsoft Windows [version 10.0.19045.5487]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\sarah_mangia\Downloads>java --version
java 17.0.12 2024-07-16 LTS
Java(TM) SE Runtime Environment (build 17.0.12+8-LTS-286)
Java HotSpot(TM) 64-bit Server VM (build 17.0.12+8-LTS-286, mixed mode, sharing)

C:\Users\sarah_mangia\Downloads>java -jar jenkins.war
Running from: C:\Users\sarah_mangia\Downloads\jenkins.war
WARNING: C:\Users\sarah_mangia\Downloads\jenkins.war
2025-03-24 07:33:53.311+0000 [id=1] INFO winstone.Logger@logInternal: Beginning extraction from war file
2025-03-24 07:33:53.452+0000 [id=1] WARNING o.e.j.w.nested.ContextHandler#setContextPath: Empty contextPath
2025-03-24 07:33:53.501+0000 [id=1] INFO org.eclipse.jetty.server.Server#doStart: Jetty-12.0.16; built: 2024-12-09T21:02:54.535Z; git: c3f80bf4e393f2308dc14dc37b942b84db0c7; jvm 17.0.12+8-LTS-286
2025-03-24 07:33:53.540+0000 [id=1] INFO o.e.j.w.s.StandardDescriptorProcessor#writeItemlet: NO JSP Support for /, did not find org.eclipse.jetty.ee9.jsp.JettyJspServlet
2025-03-24 07:33:53.542+0000 [id=1] INFO o.e.j.s.DefaultSessionManager#doStart: Session workerName=mode0
2025-03-24 07:33:56.374+0000 [id=1] INFO Hudson.WebAppMain#contextInitialized: Jenkins home directory: C:\Users\sarah_mangia\jenkins found at: $user.home/.jenkins
2025-03-24 07:33:56.880+0000 [id=1] INFO o.e.j.s.handler.ContextHandler#doStart: Started o.e.j.s.handler.ContextHandlerCoreContextHandler@1535ddfc{Jenkins v2.492.2,/,b=file:///C:/Users/sarah_mangia/.jenkins/war,a=AVAILABLE,h=org.eclipse.jetty.servlet.ServletContextHandler}
2025-03-24 07:33:56.847+0000 [id=1] INFO o.e.j.s.server.AbstractConnector#doStart: Started ServerConnector@70f43b45{HTTP/1.1,(http://1.1.1.1:8080)}
2025-03-24 07:33:56.906+0000 [id=1] INFO org.eclipse.jetty.server.Server#doStart: Started org.eclipse.jetty.server.Server@1869{STARTING[12.0.16,store=]@4751ms}
2025-03-24 07:33:56.909+0000 [id=26] INFO winstone.Logger@logInternal: Winstone Servlet Engine running: controlPort=disabled
2025-03-24 07:33:57.191+0000 [id=25] INFO Jenkins.model.Jenkins#<init>: Starting version 2.492.2
2025-03-24 07:33:57.379+0000 [id=32] INFO Jenkins.InitReactorRunner$1#onAttained: Started initialization
2025-03-24 07:33:58.150+0000 [id=37] INFO Jenkins.InitReactorRunner$1#onAttained: Listed all plugins
2025-03-24 07:34:04.563+0000 [id=36] INFO Jenkins.InitReactorRunner$1#onAttained: Prepared all plugins
2025-03-24 07:34:04.657+0000 [id=31] INFO Jenkins.InitReactorRunner$1#onAttained: Started all plugins
2025-03-24 07:34:04.667+0000 [id=34] INFO Jenkins.InitReactorRunner$1#onAttained: Augmented all extensions
2025-03-24 07:34:05.150+0000 [id=38] INFO Jenkins.InitReactorRunner$1#onAttained: Loaded all jobs
2025-03-24 07:34:07.696+0000 [id=38] INFO Jenkins.InitReactorRunner$1#onAttained: System config loaded
2025-03-24 07:34:07.741+0000 [id=36] INFO Jenkins.InitReactorRunner$1#onAttained: System config adapted
2025-03-24 07:34:07.929+0000 [id=33] INFO Jenkins.InitReactorRunner$1#onAttained: Loaded all jobs
2025-03-24 07:34:07.962+0000 [id=33] INFO Jenkins.InitReactorRunner$1#onAttained: Configuration for all jobs updated
2025-03-24 07:34:08.822+0000 [id=52] INFO Hudson.Util.Retrier#start: Attempt #1 to do the action check updates server
2025-03-24 07:34:08.822+0000 [id=34] INFO Jenkins.InitReactorRunner$1#onAttained: Completed initialization
2025-03-24 07:34:08.105+0000 [id=25] INFO Hudson.LifecycleLifecycleReady: Jenkins is fully up and running
2025-03-24 07:34:16.412+0000 [id=52] INFO h.m.DownloadService$Downloadable#load: Obtained the updated data file for hudson.tasks.Maven.MavenInstaller
2025-03-24 07:34:18.538+0000 [id=52] INFO h.m.DownloadService$Downloadable#load: Obtained the updated data file for hudson.tasks.Ant.AntInstaller
2025-03-24 07:34:19.952+0000 [id=52] INFO h.m.DownloadService$Downloadable#load: Obtained the updated data file for hudson.plugins.gradle.GradleInstaller
2025-03-24 07:34:19.952+0000 [id=52] INFO Hudson.Util.Retrier#start: Performed the action check updates server successfully at the attempt #1
```

Figure 18 : lancement du Jenkins.war

← → ↻ localhost:8080/securityrealm/firstUser

Jenkins 🔍 🛡️ log in

Dashboard > Jenkins' own user database > Create First Admin User

Create First Admin User

Username

Password

Confirm password

Full name

E-mail address

F

Une fois cela confirmé nous allons être redirigé sur la page d'accueil ;

Une fois le dépôt central créé, les développeurs devront installer un client git et le configurer avec leurs identifiants ensuite devront cloner le dépôt en suivant la commande suivante ;

Une fois la configuration de git terminée, ils vont cloner le dépôt central. Pour effectuer le clonage d'un dépôt en local il existe plusieurs techniques :

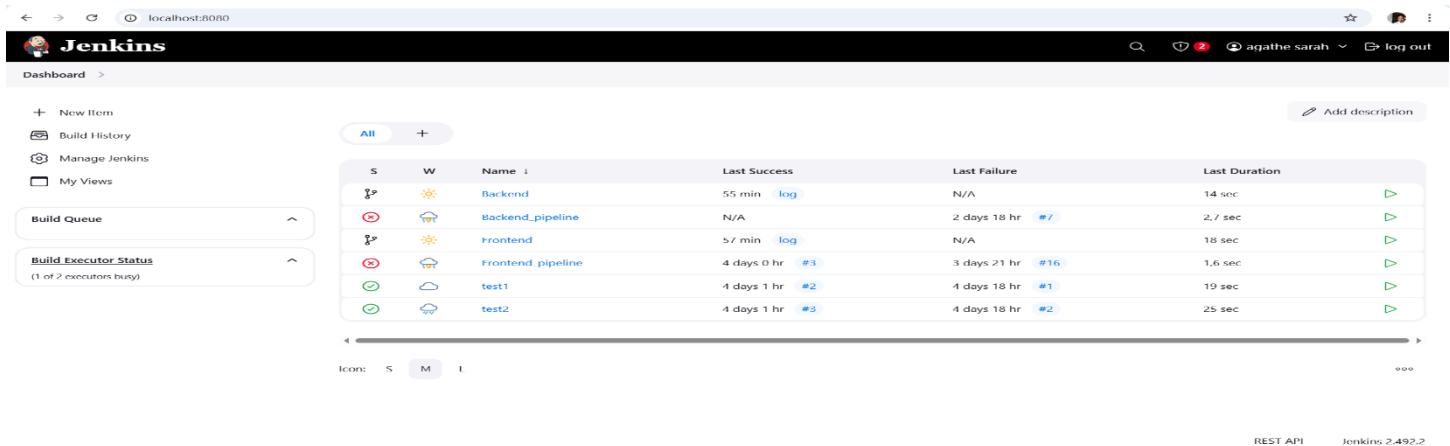


Figure 20: Interface de Jenkins

```
C:\Users\sarah.mangia\Desktop\test5>git config user.name "Joyce"
C:\Users\sarah.mangia\Desktop\test5>git config user.email "Joyce@gmail.com"
C:\Users\sarah.mangia\Desktop\test5>config user.name
'config' n'est pas reconnu en tant que commande interne
ou externe, un programme exécutable ou un fichier de commandes.
C:\Users\sarah.mangia\Desktop\test5>git config user.name
Joyce
C:\Users\sarah.mangia\Desktop\test5>git config user.email
Joyce@gmail.com
C:\Users\sarah.mangia\Desktop\test5>
```

Figure 21: Création d'identifiants git

- Par un partage **ssh** ou les développeurs devront générer leurs clés publiques et les donner au responsable du dépôt central qui devra leurs ajouter dans la liste des clés du serveur **OPEN SSH**.
- Par la méthode **samba** ou le responsable devra donner accès de partage au dépôt central et donner l'accès de lecture et d'écriture aux développeurs concerné. Le responsable devra copier le lien de partage et donner ce lien aux développeurs pour qu'ils puis le cloner dans leurs dossiers.

```
C:\Users\sarah.mangia>git clone \\YVDI1260\accounty test1
```

Figure 22: Clonage du depot central

Une fois le dépôt central cloner les développeurs peuvent maintenant ajouter de nouvelles fonctionnalités en suivant les étapes suivantes ;

```
C:\Users\sarah.mangia\Desktop\test5\backend>git checkout -b feature/fonctionalite_api  
Switched to a new branch 'feature/fonctionalite_api'  
  
C:\Users\sarah.mangia\Desktop\test5\backend>
```

Figure 23: Ajout d'une fonctionnalité

Du côté du serveur, nous allons créer un job (tache) qui nous permettra d'implémenter le pipeline CI/CD. Pour réaliser cette approche nous allons nous rendre dans l'interface de Jenkins et cliquer sur l'onglet Item :

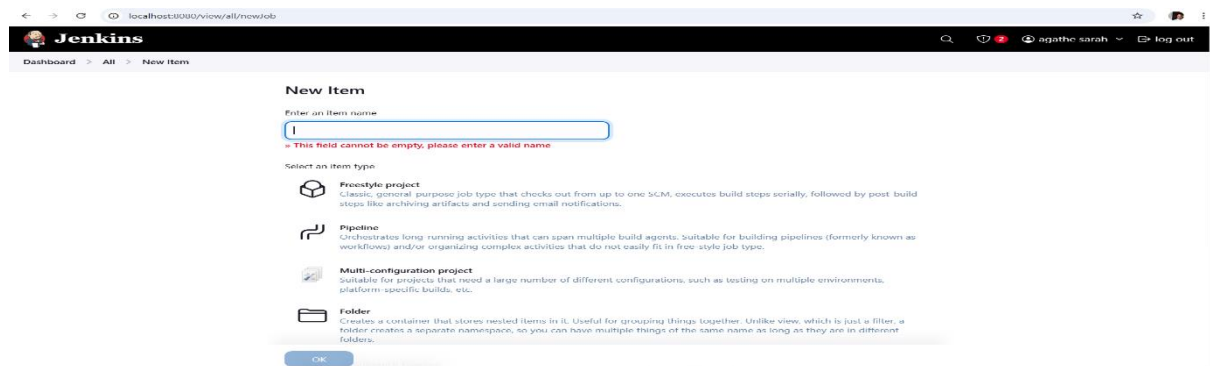


Figure 24: Création d'un job (tache)

Nous allons donner un nom et choisir le type de job à utiliser pour notre cas nous allons utiliser le type **PIPELINE** et cliquer sur **OK** et le configurer.

La configuration du job ici est tout simplement les étapes d'Intégration Continu (les builds et les tests) et du Déploiement Continu. Une fois le job crée, nous allons configurer le job ;

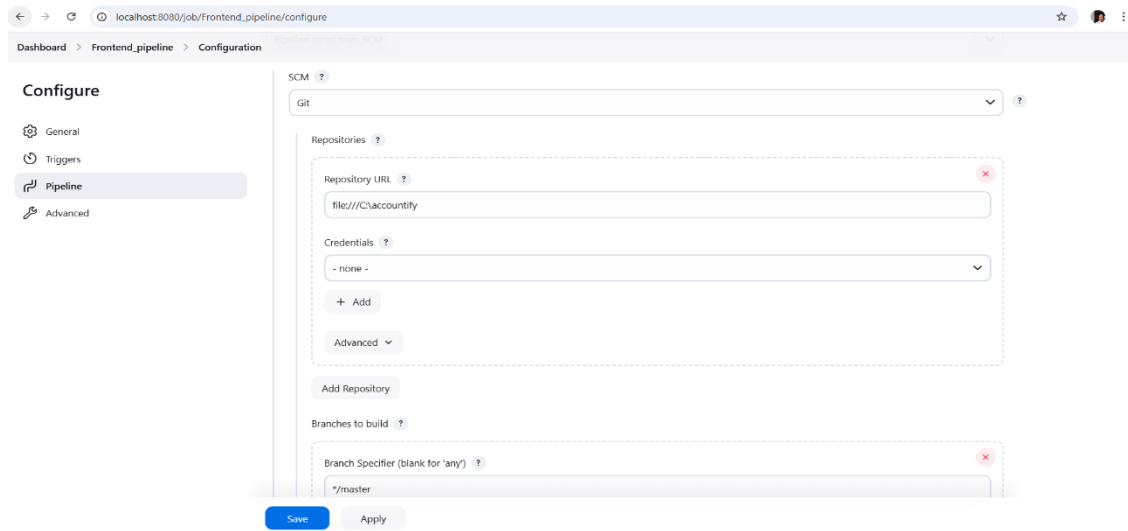


Figure 25: Configuration de communication entre git et Jenkins

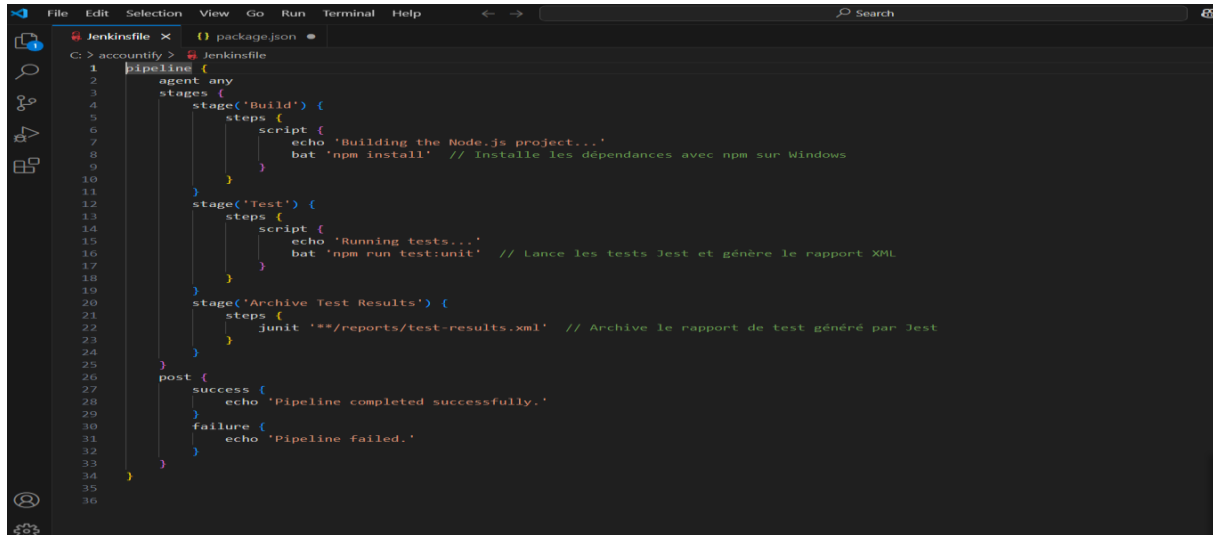
Sur cette image, pour configurer le job, nous allons ajouter l'URL de notre dépôt, mentionner la branche sur laquelle il fonctionne, ajouter un fichier Jenkinsfile et éventuellement configurer des déclencheurs (triggers).

NB : Le Jenkinsfile est un fichier écrit en Groovy (un Java simplifié) qui définit un pipeline Jenkins sous forme de code, permettant d'automatiser les étapes de compilation (build), de test (CI) et de déploiement (CD). Il peut être écrit en deux styles : Scripted ou Declarative. Le choix du style à utiliser dépendra de nos besoins. Le tableau suivant nous aidera à déterminer quel est le choix le plus approprié.

Tableau 3 : Détermination sur le choix du langage pipeline à prendre

Critères	Jenkins Declarative Pipeline	Jenkins Scripted Pipeline
Simplicité	Plus simple et plus lisible	Plus complexe et verbeux
Syntaxe	Basé sur une structure claire et stricte	Plus flexible mais plus difficile à maîtriser
Facilité de maintenance	Facile à lire et à maintenir	Peut devenir compliqué avec le temps
Gestion des erreurs	Gestion automatique des erreurs et des étapes post-actions	Doit être géré manuellement avec <code>try-catch</code>
Flexibilité	Moins flexible (moins de personnalisation possible)	Très flexible, permet du Groovy avancé
Adoption recommandée	Recommandé pour la plupart des projets	Recommandé pour des besoins très avancés
Support et documentation	Mieux documenté et plus standardisé	Moins documenté, nécessite plus de connaissances en Groovy

Nous pouvons donc déduire de ce tableau qu'écrire un Jenkinsfile en langage déclaratif est le plus recommandé, car il est simple, facile à maintenir et moins sujet aux erreurs.



```
1 pipeline {
2   agent any
3   stages {
4     stage('Build') {
5       steps {
6         script {
7           echo 'Building the Node.js project...'
8           bat 'npm install' // Installe les dépendances avec npm sur Windows
9         }
10      }
11    }
12    stage('Test') {
13      steps {
14        script {
15          echo 'Running tests...'
16          bat 'npm run test:unit' // Lance les tests Jest et génère le rapport XML
17        }
18      }
19    }
20    stage('Archive Test Results') {
21      steps {
22        junit '**/reports/test-results.xml' // Archive le rapport de test généré par Jest
23      }
24    }
25  }
26  post {
27    success {
28      echo 'Pipeline completed successfully.'
29    }
30    failure {
31      echo 'Pipeline failed.'
32    }
33  }
34 }
35
36
```

Figure 26: Jenkinsfile

Ce fichier Jenkinsfile doit être placé à la racine du dépôt central. Ensuite, enregistrez les modifications et lancez le premier build. Cela permettra à Jenkins de communiquer avec Git, de récupérer le dépôt et d'effectuer un build. Un Jenkinsfile ressemble généralement à :

Ici, nous avons un Jenkinsfile écrit en langage déclaratif, où sont définies les différentes étapes. Notre Jenkinsfile se limite uniquement aux tests, car nous voulons d'abord évaluer son comportement. Nous allons détailler les différentes étapes de ce Jenkinsfile.

- Pipeline : sert de point de départ du Jenkinsfile ce qui permet de déclarer un pipeline complet et il contient tous les autres éléments (stages, steps, et post)
- Agent any : Décrire simplement l'environnement dans lequel le pipeline sera exécuter et ajouter any, signifie que le pipeline sera exécuté dans d'importe quel nœud dans l'environnement (ex ; Windows, linux, docker etc...).
- Stages : représente les parties du pipeline.
- Steps : les actions à effectuer.
- Post : pour les résultats du pipeline.

Une fois le Jenkinsfile écrit et enregistré à la racine du dépôt, nous pouvons revenir dans Jenkins et lancer le build. Ensuite, nous pourrons voir le résultat de notre build sous l'onglet 'Stages', où ce résultat s'affichera.

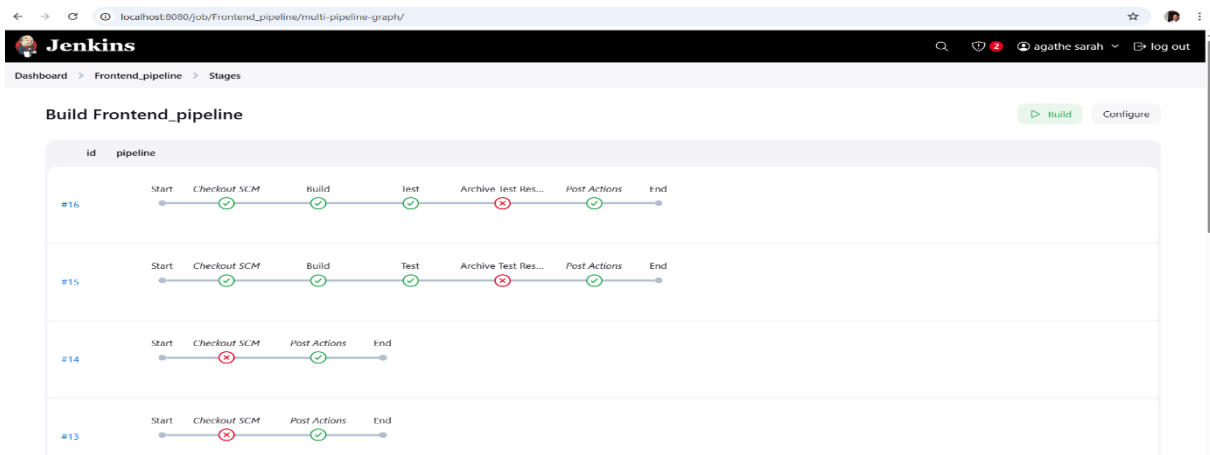
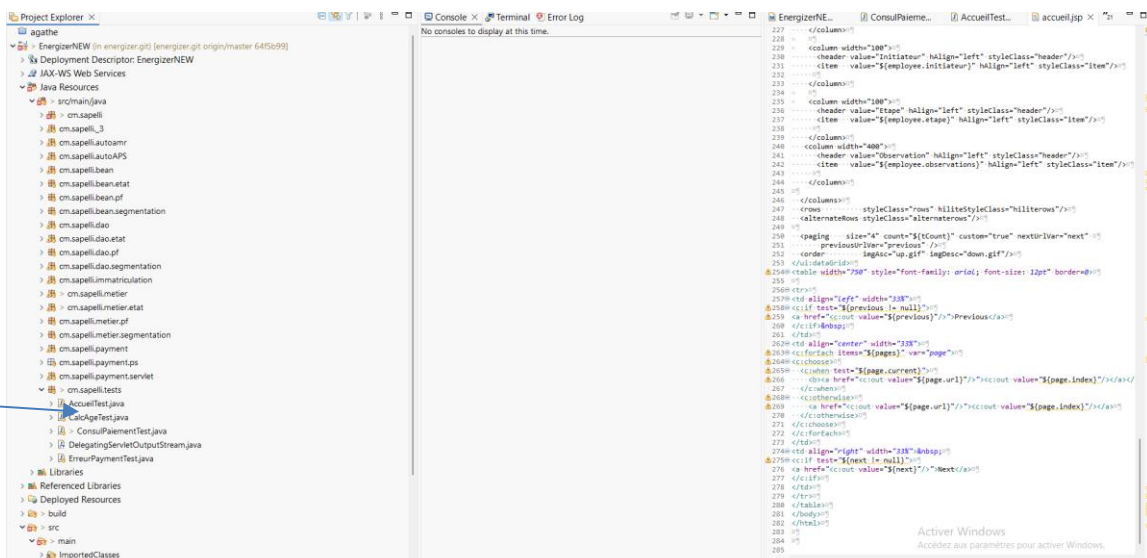


Figure 27: Détail du pipeline

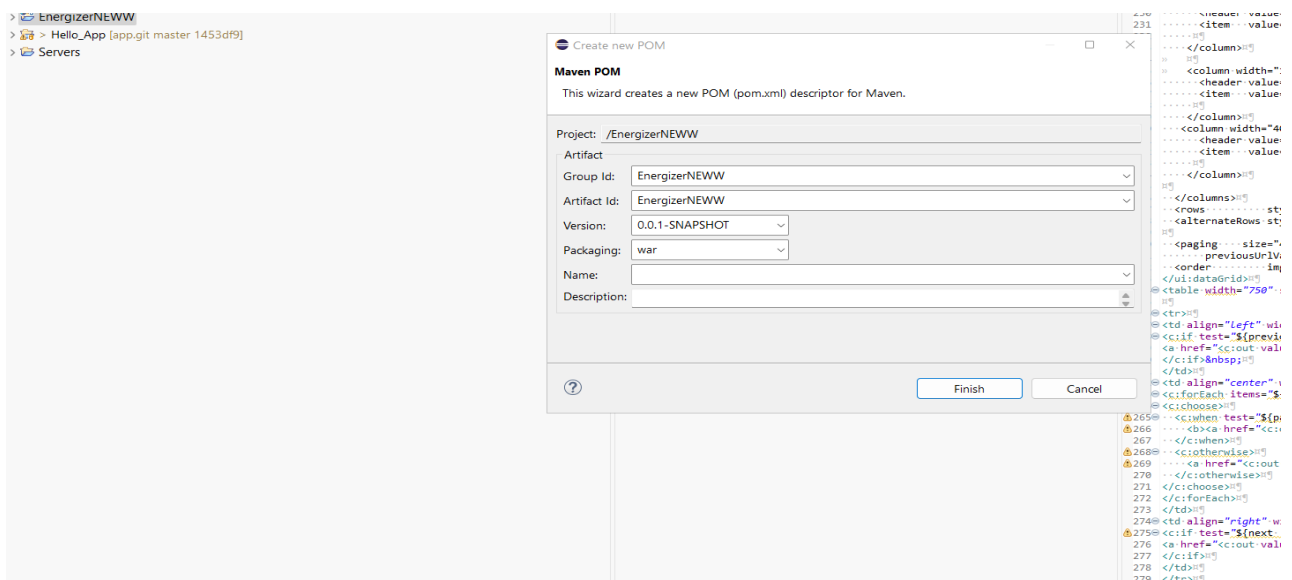
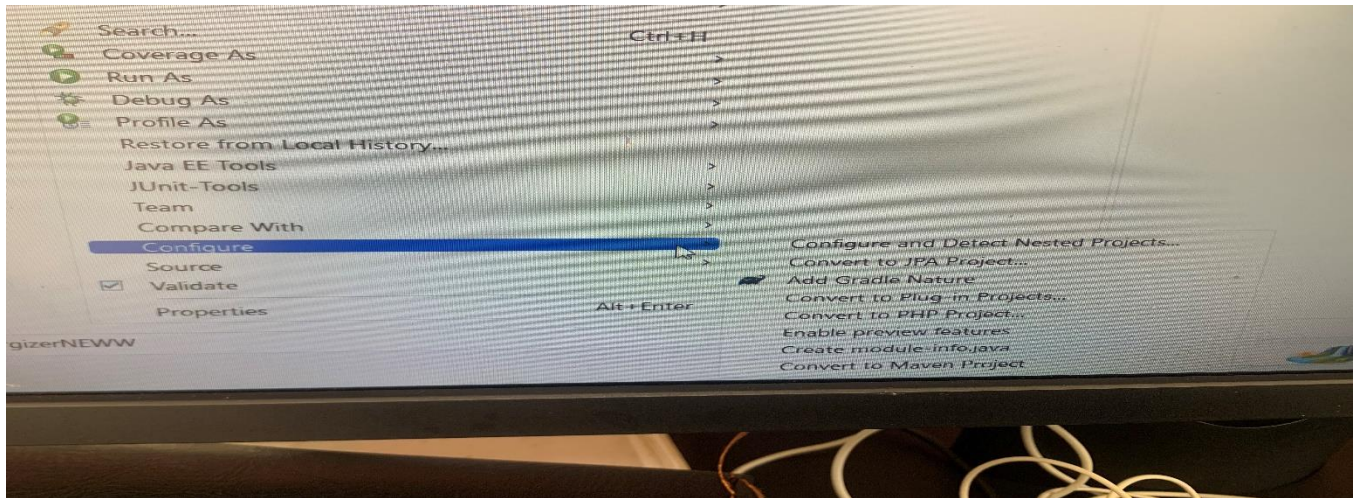
Ici nous pouvons voir les différentes étapes du pipeline où ; les builds (checkout du code source a été récupéré, la compilation du code aussi mais il y'a des problèmes aux niveaux des tests ce qui ne permet pas à l'application d'aller en déploiement.

Pourquoi les tests ne s'exécutent pas ?

- L'échec de lancement des tests vient simplement de l'absence d'un dossier destiné aux tests qui cause cette erreur. Pour corriger l'erreur, nous allons revoir le code en ajoutant un dossier dédié aux tests.



Nous avons ajouté un dossier dédié aux tests que nous avons exécuté à l'aide des Framework de tests comme JUNIT, MOCKITO. Une fois cela fait, nous avons relancer le pipeline et pour que JENKINS exécute les tests automatiquement, il le fait par Maven et c'est ainsi que nous avons converti le projet java en un projet maven ;



Une fois le projet java converti, un fichier pom.xml (fichier contenant toute les dépendances nécessaire) sera généré. Ensuite nous avons exécuté les tests à l'aide d'une commande Maven, (mvn test), ce qui va permettre à maven, d'aller chercher le dossier destiné au test.

Pourquoi avoir choisi Maven ?

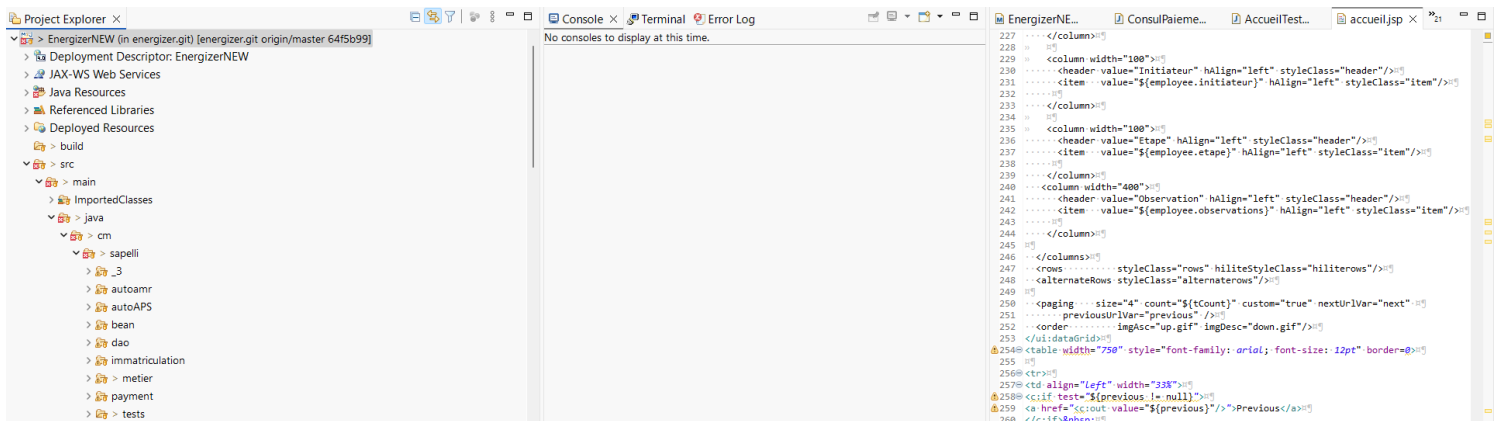
- Etant donné que nous avons effectué une étude sur le code existant et de son architecture,

nous avons remarqué que le code en lui-même avait besoin d'une re-structuration et Maven fournit donc cette restructuration en reorganisant les fichiers.

- Pour une reorganistaion car l'organisation du code est confuse, de type "spaghetti", avec un mélange entre la logique métier et la couche de présentation (Vue).

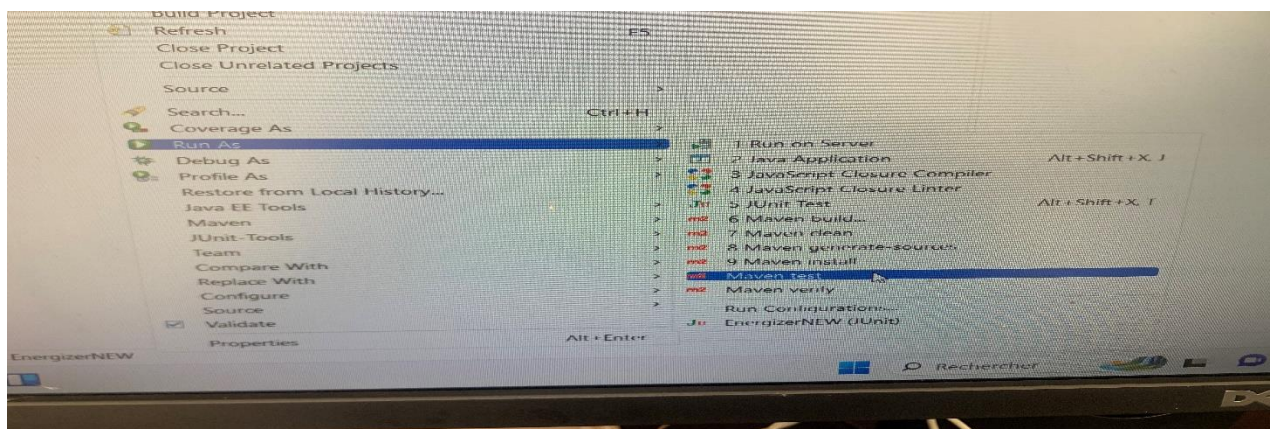
Pourquoi inclure les tests dans le code ?

- L'écriture des tests dans du code est une phase essentielle dans le developpement d'une application car ces tests veille sur la performance, robustesse, stabilité et le comportement attendu de l'application avant la mise en production.



C'est ainsi que nous avons d'abord voulu exécuter les tests dans un premier temps sous notre IDE.

Les tests sont des tests unitaires et d'intégration sur les différentes fonctionnalités de l'application.



Les tests exécuté, et nous avons consulté la console qui nous a afficher l'echec des tests ;

```

terminated> C:\Program Files\Java\jdk-11\bin\java.exe (20 juin 2023, 06:40:53) [pid: 3064]
[DEBUG] Imported: org.eclipse.aether.artifact < plexus.core
[DEBUG] Imported: org.eclipse.aether.collection < plexus.core
[DEBUG] Imported: org.eclipse.aether.deployment < plexus.core
[DEBUG] Imported: org.eclipse.aether.graph < plexus.core
[DEBUG] Imported: org.eclipse.aether.impl < plexus.core
[DEBUG] Imported: org.eclipse.aether.installation < plexus.core
[DEBUG] Imported: org.eclipse.aether.internal.impl < plexus.core
[DEBUG] Imported: org.eclipse.aether.metadata < plexus.core
[DEBUG] Imported: org.eclipse.aether.repository < plexus.core
[DEBUG] Imported: org.eclipse.aether.resolution < plexus.core
[DEBUG] Imported: org.eclipse.aether.spi < plexus.core
[DEBUG] Imported: org.eclipse.aether.transfer < plexus.core
[DEBUG] Imported: org.eclipse.aether.util < plexus.core
[DEBUG] Imported: org.eclipse.aether.version < plexus.core
[DEBUG] Imported: org.fusesource.jansi.* < plexus.core
[DEBUG] Imported: org.slf4j.* < plexus.core
[DEBUG] Imported: org.slf4j.event.* < plexus.core
[DEBUG] Imported: org.slf4j.helpers.* < plexus.core
[DEBUG] Imported: org.slf4j.spi.* < plexus.core
[DEBUG] Populating class realm maven.api
[DEBUG] Created adapter factory; available factories [file-lock, rwlock-local, semaphore-local, noop]; available name mappers [discriminating, file-]
[INFO] Error stacktraces are turned on.
[DEBUG] Message scheme: color
[DEBUG] Message styles: debug info warning error success failure strong mojo project
[DEBUG] Reading global settings from settings.xml
[DEBUG] Reading user settings from C:\Users\sarah.mangia\.m2\settings.xml
[DEBUG] Reading global toolchains from toolchains.xml
[DEBUG] Reading user toolchains from C:\Users\sarah.mangia\.m2\toolchains.xml
[DEBUG] Using local repository at C:\Users\sarah.mangia\.m2\repository
[DEBUG] Using manager EnhancedLocalRepositoryManager with priority 10.0 for C:\Users\sarah.mangia\.m2\repository
[INFO] Scanning for projects...
[DEBUG] Extension realms for project EnergizerNEW:EnergizerNEW:war:0.0.1-SNAPSHOT: (none)
[DEBUG] Looking up lifecycle mappings for packaging war from ClassRealm[plexus.core, parent: null]
[DEBUG] === REACTOR BUILD PLAN ======================================
[DEBUG] Project: EnergizerNEW:EnergizerNEW:war:0.0.1-SNAPSHOT
[DEBUG] Tasks: [test]
[DEBUG] Style: Regular
[DEBUG] =====
[INFO] -----< EnergizerNEW:EnergizerNEW >-----
[INFO] Building Energizer 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ war ]-----
[DEBUG] Lifecycle clean -> [pre-clean, clean, post-clean]
[DEBUG] Lifecycle default -> [validate, initialize, generate-sources, process-sources, generate-resources, process-resources, compile, process-class]
[DEBUG] Lifecycle site -> [pre-site, site, post-site, site-deploy]
[DEBUG] Creating adapter using NameMapper 'gav' and factory 'rwlock-local'
[DEBUG] Resolving artifact org.apache.maven.plugins:maven-surefire-plugin:pom:3.5.0 from [central (https://repo.maven.apache.org/maven2, default, re
[WARNING] The POM for org.apache.maven.plugins:maven-surefire-plugin:jar:3.5.0 is missing, no dependency information available
[DEBUG] Resolving artifact org.apache.maven.plugins:maven-surefire-plugin:jar:3.5.0 from [central (https://repo.maven.apache.org/maven2, default, re
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 0.433
[INFO] Finished at: 2025-06-20T08:40:57+01:00
[INFO] -----
[ERROR] Plugin org.apache.maven.plugins:maven-surefire-plugin:3.5.0 or one of its dependencies could not be resolved:
[ERROR] Cannot access central (https://repo.maven.apache.org/maven2) in offline mode and the artifact org.apache.maven.plugins:maven-surefir

```

Pourquoi les tests ont échoué malgré la transition vers un projet Maven ?

Les tests échouent tout simplement à cause de certaines dépendances qui sont obsolète car ils datent de depuis.

- Nous avons aussi remarqué que l'application tournait sur des JSP, et qui eux aussi sont aujourd'hui obsolète et ont été remplacé par Thymeleaf qui lui est un moteur de template HTML dans les applications Java avec Spring Boot qui vont permettre de créer des pages web dynamique en gardent un HTML propre, lisible. L'application présente des complications dues à l'utilisation de dépendances obsolètes.
- L'organisation du code est confuse, de type "spaghetti", avec un mélange entre la logique métier et la couche de présentation (Vue).
- Plusieurs pages JSP ne s'exécutent plus.
- Des erreurs sont liées à des éléments obsolètes dans le fichier `web.xml`.
- Les technologies utilisées (Tomcat, Java, bibliothèques, etc.) ont évolué et nécessitent des mises à jour.

- Les JSP sont aujourd'hui considérées comme obsolètes dans la majorité des projets Web modernes en Java.

Pour pouvoir résoudre ce blocage, nous avons effectué une étude de correction et avons ressorti une Analyse et des suggestions ;

Analyse

Constat Général

Suite à l'analyse de l'application existante, plusieurs constats importants ont été faits concernant son comportement et sa structure technique :

Remarque :

- L'application présente des complications dues à l'utilisation de dépendances obsolètes.
- L'organisation du code est confuse, de type "spaghetti", avec un mélange entre la logique métier et la couche de présentation (Vue).
- Plusieurs pages JSP ne s'exécutent plus.
- Des erreurs sont liées à des éléments obsolètes dans le fichier `web.xml`.
- Les technologies utilisées (Tomcat, Java, bibliothèques, etc.) ont évolué et nécessitent des mises à jour.
- Les JSP sont aujourd'hui considérées comme obsolètes dans la majorité des projets Web modernes en Java.

Recommandations

Pour remédier à ces problèmes et moderniser l'application, deux grandes options s'offrent à nous :

Option A : Modernise le monolithe existant

Cette option vise à refactoriser l'application en gardant une seule base de code, un seul déploiement mais en respectant les bonnes pratiques : MVC, API REST, découpage Vue/Backend, tests, CICD. C'est-à-dire :

Avant (Monolithe classique JSP/Servlet)	Après modernisation (Monolithe MVC moderne)
Vue, logique métier, accès BD dans la même JSP	Séparation claire : Vue= React, Backend+ Spring Boot.
Aucune API REST	Backend expose des endpoints REST accessibles
Code couplé et difficile à tester	Code modulaire, testable, évolutif
Déploiement manuel	Pipeline CICD/automatisé
Pas de séparation claire des responsabilités	Architecture MVC +API

Pourquoi moderniser le monolithe ?

a. objectif

- On garde la même logique existante.
- Préparer la migration vers les micros services, en modernisant le monolithe, il sera facile d'identifier les modules logiques, sans casser tout le code.
- Faciliter les tests automatisés état donné que le code sera modulaire, il sera plus évident d'écrire les tests.
- Améliorer la maintenabilité, le code sera plus propre, organisé par responsabilité (MVC).
- Pont vers le CICD, il sera possible d'automatiser les tests, builds et le déploiement.

b. Résultat après la modernisation

On est toujours dans une application monolithique, mais :

- Séparée en couches bien définies (MVC)
- Facile à maintenir et à tester
- Déployable automatiquement
- Prêt pour accueillir des micro services.

Option B : Migration vers une architecture Micro service

a. objectif

- Scalabilité indépendante. Dans les micro services, chaque service peut être déployé et géré séparément.
- Indépendance des équipes. Une équipe peut développer et déployer un micro service sans impacter les autres.
- Déploiement plus rapide. Les micro services peuvent être mis à jour sans redéployer toute l'application.
- Résilience accrue. Si un service tombe, les autres peuvent continuer à fonctionner.
- Technologies variées. Chaque service peut utiliser le langage ou la base de données le mieux adapté.

NB : les micro services sont une préparation au cloud (idéale pour Docker, Kubernetes etc.)

Certes cette approche serait l'idéale mais peut être risqué si nous devons directement quitter du Monolithe au Micro services.

b. Failles/Risques

Failles/Risques	Raisons
Exposition de complexité	On quitte un seul déploiement à une dizaine ce qui demandera plus de code, plus de configurations, et des erreurs potentiels.
Perte de logique Métier	En cas d'incompréhension de la monolithe existant, le risque de casser et l'oubli des règles importantes peut arriver.
Complexité des tests	Chaque service doit être testé et l'interactions entre eux.
Déploiement plus lourd	Gestion des ports, sécurité entre services, réseaux.
Surcoût technique	API Gateway, CICD, Monitoring, Docker doit être mis en place

NB : Quand une application est bien découpée, bien testée, et stable en monolithe moderne, alors là nous pouvons parfaitement migrer.

Justification des Recommandations

Les recommandations sont motivées par plusieurs objectifs techniques et stratégiques :

- Passage à une chaîne de développement moderne intégrant **CI/CD** (tests unitaires, intégration, régression, end-to-end, etc.).
- Utilisation de **Maven** pour la gestion des dépendances et l'automatisation du build.
- Intégration future avec des outils comme **Jenkins**, qui utilisent Maven pour automatiser les tests via une seule commande.
- La conversion initiale vers un projet Maven a permis une meilleure organisation, mais a révélé des problèmes de
- Compatibilité avec les dépendances obsolètes, rendant la maintenance difficile.

Etude comparative entre une architecture Monolithe existante, monolithe modernisé et Micro services

Critères	Monolithe	Monolithe Moderne	Micro services
Structure	Tout en un seul bloc	Séparation claire (MVC, Front, Back, REST)	Service indépendants
Déploiement	Un seul fichier WAR/JAR	Un JAR + un Build React (02 artefacts)	Plusieurs services déployés séparément
Technologie	Java EE (JSP, Servlets, JDBC)	Spring Boot, React, JPA, REST	Spring Boot , REST , Kafka, API Gateway
Maintenance	Difficile, code couplé	Facile, structure modulaire	Plus complexe, nécessite une orchestration
Tests	Difficile (pas de séparation)	Tests unitaires + intégration facile	Nécessite tests interservices +mocks
CICD	Difficile à intégrer	Facile (React +Maven pipeline)	Complexe (pipeline Multi-services)
Sécurité	Centralisée mais basique	Sécurisation REST (JWT, OAuth)	Besoin de Gateway, tokens, isolation réseau
Base de données	Une seule BD	Une seule BD utilisée via JPA/JDBC	Chaque service a sa propre BD
Responsabilité métier	Non définie (mélangée dans JSP)	Séparée (service, controller, repository)	Isolée
Refactoring progressif possible	Difficile	Oui (module par module)	Oui
Temps de mise en œuvre	Court au début	Moyen	Long
Évolutivité	Limitée	Bonne, organisé	Très élevée

Couplage des modules	Très fort	Faible	Nul (autonomie de chaque services)
Courbe d'apprentissage	Faible (JSP simple)	Moyenne (spring/React)	Forte (dev, infrastructure , CICD, cloud)

Architecture Monolithe (architecture actuel)

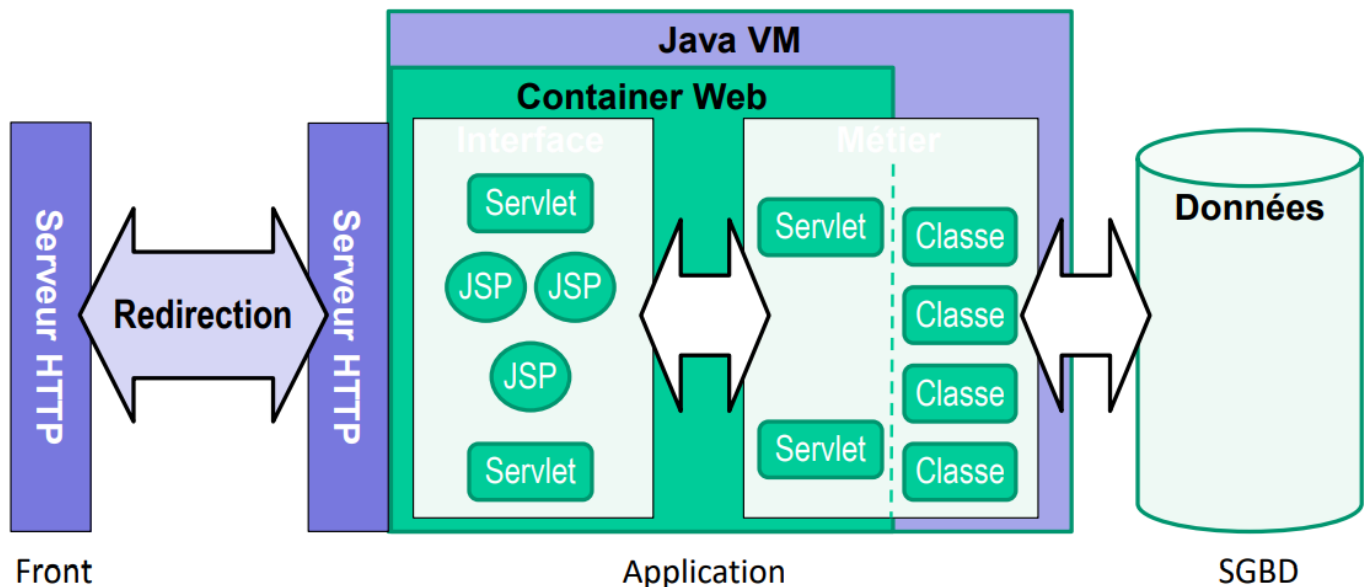


Figure 28:. Architecture Monolithe

Ici est représentée l'architecture actuelle répartie en trois couches (03) :

- **Package Java** : contient les **modules métiers**, les **services** et les **DAO** pour la gestion de la logique métier et de l'accès aux données.
- **Servlets** : jouent le rôle de **contrôleurs**, responsables de la **réception des requêtes**, de l'**appel au modèle** (services, DAO), et de la **redirection vers les pages JSP**.
- **JSP** : représentent les **pages web côté serveur**, jouant le rôle de **vue**, qui affichent dynamiquement des données à l'aide de **HTML** et de **code Java intégré**.

Dans l'architecture monolithique, ces trois couches sont **intégrées dans une seule application** déployée et maintenue de manière centralisée.

Tous les composants (vue, contrôleur, modèle, accès aux données) sont **étroitement couplés** et **déployés ensemble**, ce qui rend l'application **simple à construire** mais **difficile à faire évoluer** ou à **scaler indépendamment**.

Architecture Monolithe Moderne (MVC)

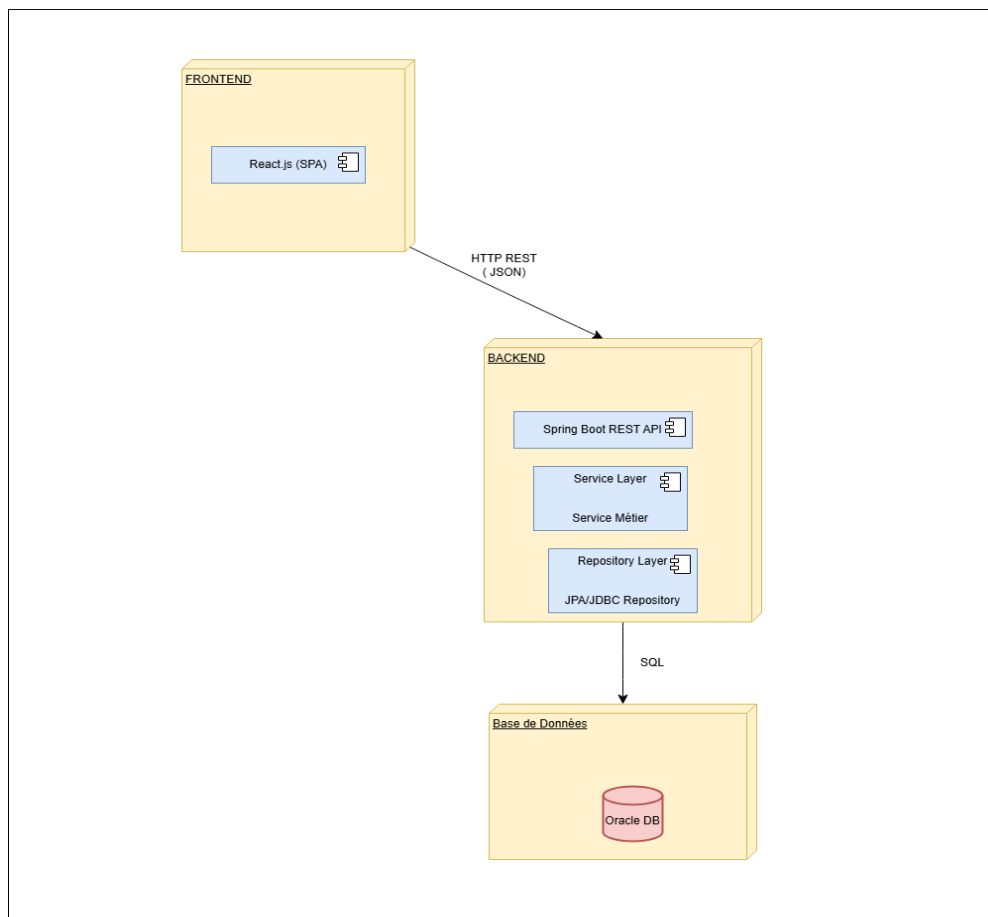


Figure 29. Architecture Monolithe Moderne

Architecture Micro-service

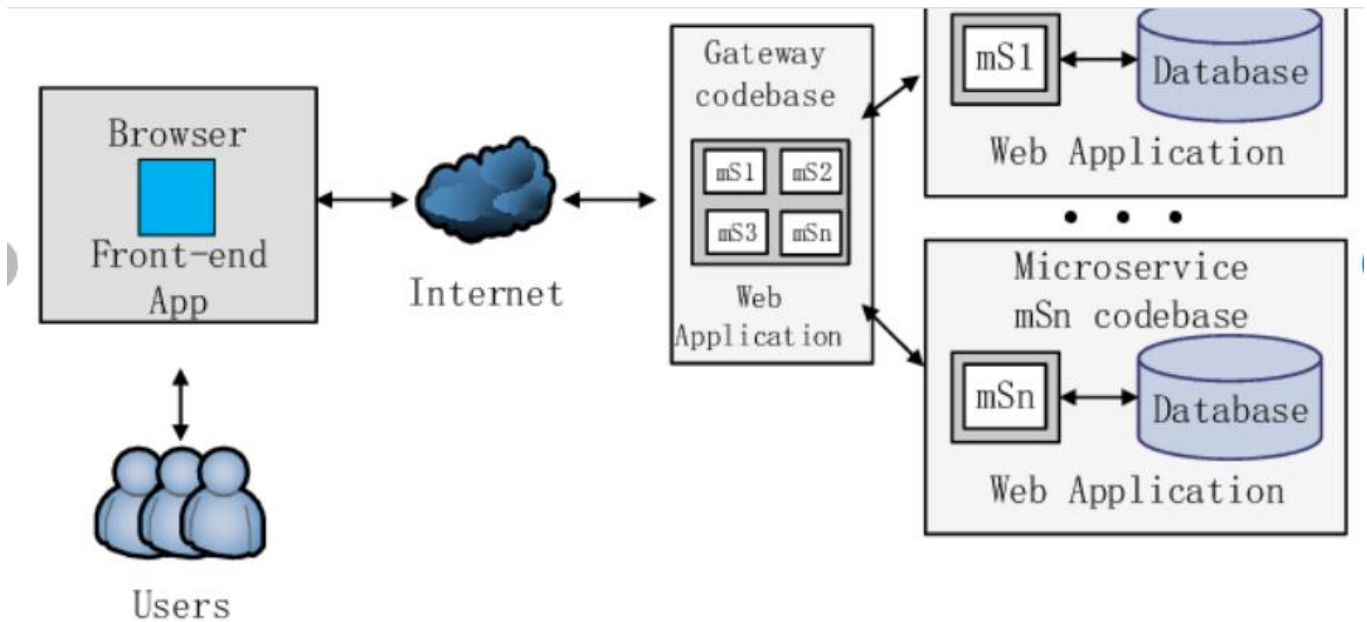


Figure 30. architecture microservice

Dans l'architecture microservices l'application est **découpée en plusieurs services indépendants**, Chacun responsable d'un domaine métier précis.

Chaque microservice dispose de **sa propre structure complète**, incluant ses **services, contrôleurs, modèles**, et parfois sa **base de données**. Chaque microservice contient ;

- **Son propre package Java** (ou code métier) : logique métier, DAO, services propres au domaine (ex. réservation, client, paiement...).
- **Un contrôleur REST** (@RestController) : qui remplace les servlets, et expose des **API REST** pour que le frontend ou d'autres services puissent interagir.
- **Plus de JSP** : le frontend est complètement **dissocié** et développé séparément (souvent en **React, Vue, Angular**, etc.). Le frontend communique uniquement avec les microservices via **HTTP (API REST)**.

PERSPECTIVES

Après avoir remplacé l'architecture monolithique classique (JSP/Servlets) par une architecture monolithique modernisée, basée sur Spring Boot côté backend et React côté frontend, nous avons planifié une démarche structurée visant à introduire le CI/CD dans le processus de développement.

Cette modernisation permettra désormais une meilleure séparation des responsabilités (MVC), une exposition claire des API REST, ainsi qu'un frontend découplé, facilitant les étapes ultérieures d'automatisation.

Les phases suivantes sont donc axées sur :

- La gestion du code source via Git ;
- L'intégration continue via Jenkins ;
- La construction automatisée des artefacts back end et frontend ;
- Et le déploiement automatisé en environnement de test.

Pour pouvoir aller jusqu'au bout de notre implémentation, nous allons refactoriser les jsp en Front/Back et maintenir la base de données. Pour cette approche, nous avons repartit les tâches comme suite ;

1.1 JSP existente

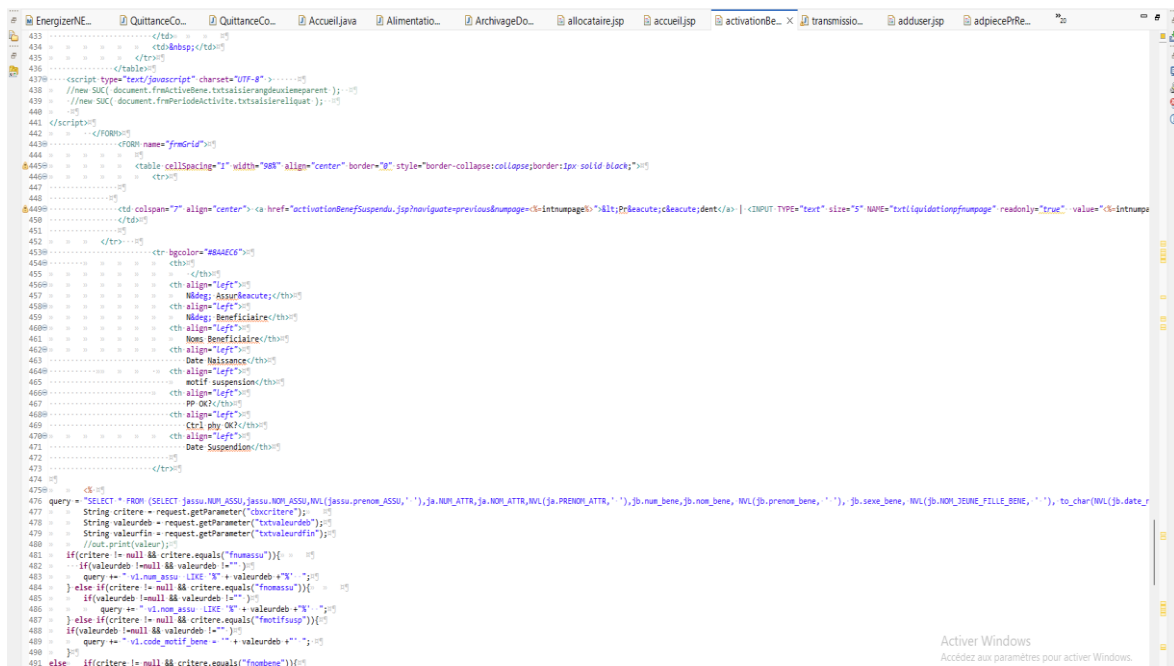
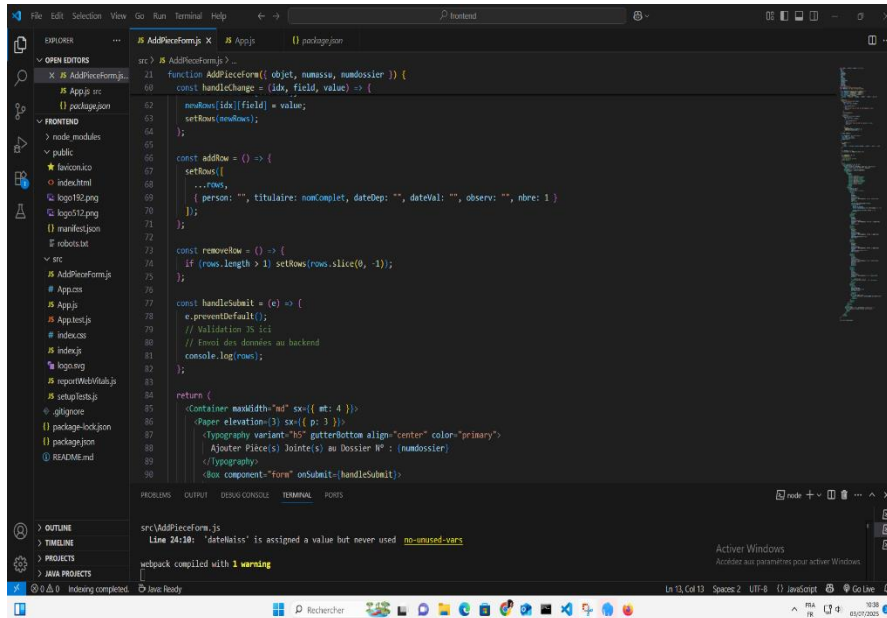


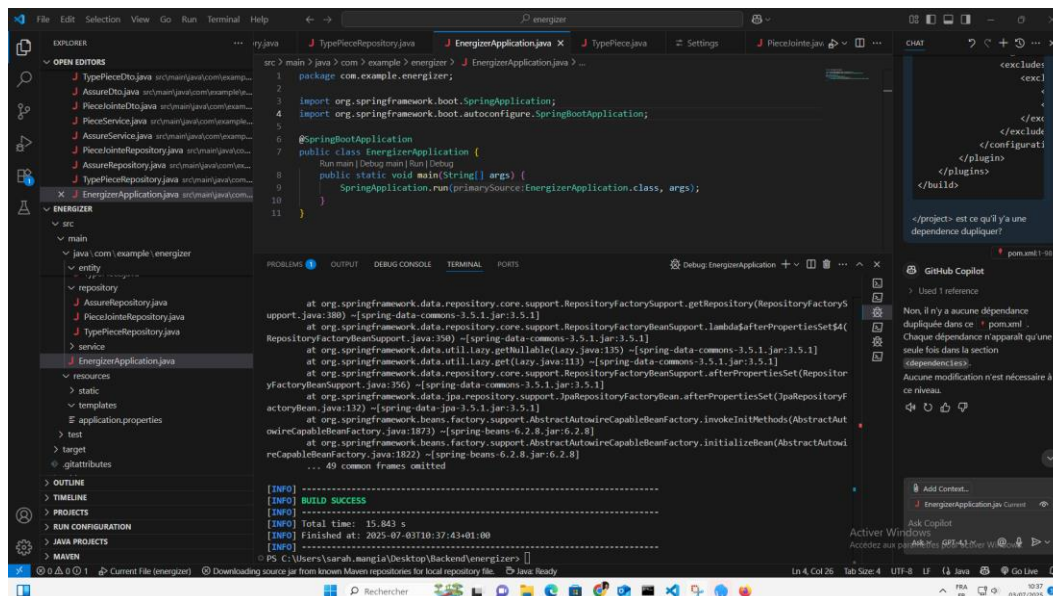
Figure 31. fichier JSP

MISE EN ŒUVRE D'UN PIPELINE D'INTÉGRATION ET DE DÉPLOIEMENT CONTINU (CI/CD) BASÉ SUR JENKINS ET GIT AU SEIN DE LA CNPS CAS DE SAPELLI ENERGIZER

Frontend



Backend



BD :

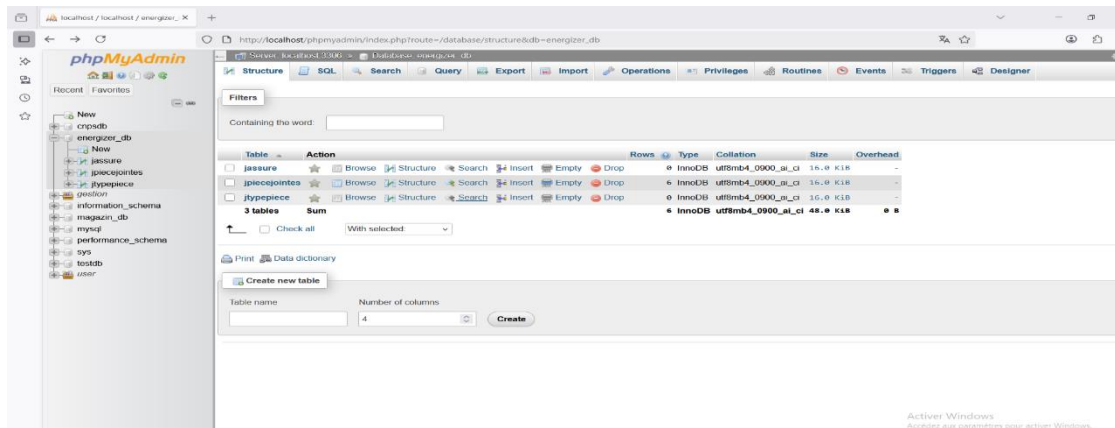


Figure 34. Base de données.

Ici est présenté la Base de données avec ses tables générées.

1.3 Présentation

Ajouter Pièce(s) Jointe(s) au Dossier N° :

#	Nature Pièce	Titulaire	Date Dépôt	Date Signature	Observations	Nb Pièce	Act
1	Nb	Tit... Ag:	Dépôt * jj/mm/aaaa	Signature * jj/mm/aaaa	Obse...	Nb 1	

VALIDER

Figure 35. presentation

- Après cette refactorisation, nous allons maintenir la Base de données existante.
- Par la suite, nous allons créer deux (02) référentiels git :
 - ✚ Un référentiel pour le frontend
 - ✚ Un référentiel pour le backend
- Ensuite nous allons créer deux (02) tâches dans Jenkins et allons configurer la connexion aux référentiels respectives :
 - ✚ Une tâche (01) pour le frontend

✚ Une tâche (01) pour le backend

- Une fois que nous aurions créé les deux tâches, nous allons configurer la connexion des référentiels au serveur ou est héberger Jenkins :
- Une fois la connexion établie, nous allons faire les premiers tests de récupération des codes.
- Etant donné que pour que Jenkins automatise les tâches (tests, builds, déploiement), son moteur doit être configuré (Jenkinsfile), en local, ce fichier doit être écrit mais Jenkins dispose des plugins qui peuvent le générer mais en cas de modification, demandera une main œuvre manuel sachant bien que nous sommes en local (une des limites de Jenkins configuré avec git en local).

```
ntify > Jenkinsfile
pipeline {
  agent any
  environment {
    FRONTEND_DIR = 'C:/Frontend'
    FRONTEND_DEPLOY_DIR = "${LARAGON_DIR}/Frontend"
  }
  stages {
    stage('Checkout Frontend') {
      steps {
        script {
          dir(FRONTEND_DIR) {
            bat 'git checkout master'
          }
        }
      }
    }

    stage('Install Dependencies') {
      steps {
        dir(FRONTEND_DIR) {
          bat 'npm install --legacy-peer-deps'
        }
      }
    }

    stage('Build Frontend') {
      steps {
        dir(FRONTEND_DIR) {
          bat 'npm run build'
        }
      }
    }

    stage('Test') {
```

Figure 36.Jenkinsfile

- Une fois ce fichier prêt, nous pouvons l'exécuter automatiquement le frontend et le backend :

Si le build réussit, nous allons juste l'exécuter en production et ce processus se fera de manière continue.

Diagramme de Flux du Processus du CICD

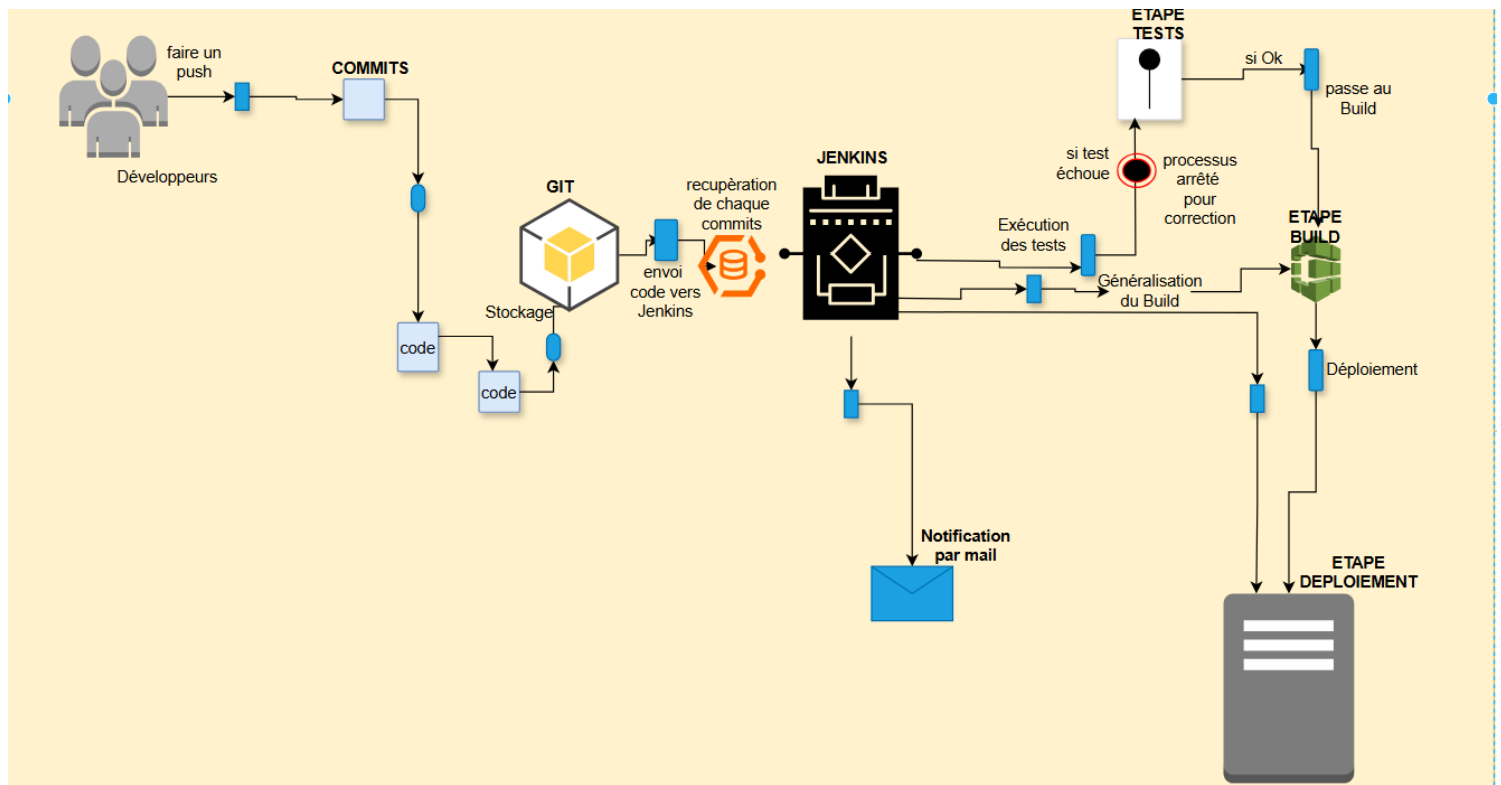


Figure 37. Flux du Processus du CICD.

CONCLUSION GENERALE

Ce stage de six mois au sein de la Caisse Nationale de Prévoyance Sociale (CNPS) m'a offert une immersion concrète dans un environnement technique structuré, au cœur des enjeux de transformation numérique d'un organisme public d'envergure. L'objectif initial, centré sur l'implémentation d'un pipeline CI/CD avec Jenkins et Git, visait à automatiser le processus de développement et de déploiement logiciel, dans une logique d'optimisation continue et de réduction des risques opérationnels. Au fil de l'avancement du projet, plusieurs jalons ont pu être atteints : la centralisation du code dans un référentiel Git unique, la configuration de Jenkins, ainsi que la mise en œuvre des étapes automatisées de compilation, de test et de déploiement. Cependant, certaines contraintes techniques — notamment la nécessité de refactorer l'architecture existante pour permettre une intégration complète du pipeline — ont conduit à reporter certaines optimisations à une phase ultérieure. En raison de ces ajustements structurels et du temps limité imparti au stage, l'intégration complète et fluide du pipeline dans l'écosystème applicatif n'a pas pu être finalisée dans son intégralité. Néanmoins, les fondations posées offrent un socle solide pour la poursuite des travaux par les équipes internes, avec une vision claire des prochaines étapes à engager. En définitive, ce stage m'a permis de renforcer mes compétences en DevOps, de mieux appréhender les enjeux liés à l'automatisation des processus de développement en entreprise, et de contribuer activement à une dynamique d'innovation au sein de la CNPS. Le travail réalisé constitue une base évolutive qui s'inscrit dans une perspective d'amélioration continue des pratiques techniques de l'institution.

ABREVIATIONS ET MOT-CLES

- **CICD** : Integration Continu et Déploiement Continu
- **Pipeline** : Chaîne automatisée d'étapes qui s'exécutent l'une après l'autre pour construire, tester, valider, et déployer une application logicielle
- **Stages** : un stage représente une étape importante du processus, comme la compilation, les tests, le déploiement, etc.
- **Monolithe** : Application unique et indivisible où toutes les fonctionnalités sont regroupées dans un seul bloc de code.
- **Micro-service** : L'architecture micro services consiste à découper une application en plusieurs petits services indépendants, chacun responsable d'une seule fonction métier.
- **Triggers** : Evénement qui déclenche une action dans un pipeline CI/CD ou dans un système de gestion de versions comme Git.
- **GIT** : Système de gestion de versions décentralisé. Il permet à plusieurs développeurs de travailler sur le même projet en suivant l'historique des modifications.
- **Jenkins** : Outil open-source d'intégration continue (CI) qui permet d'automatiser les tâches de développement logiciel
- **Build** : Processus de transformation du code source d'un projet en une application exécutable ou en un fichier déployable (comme un .jar, .exe, .apk, .zip, etc.).
- **Jenkinsfile** : Fichier de configuration écrit dans un langage de script (souvent **Groovy**), qui permet de définir un pipeline CI/CD dans Jenkins.
- **Tests** : Vérifications automatiques ou manuelles qui permettent de s'assurer que le programme fonctionne correctement, selon les attentes et les spécifications.
- **Référentiel** : est un espace de stockage où l'on conserve tout le code source d'un projet, ainsi que l'historique complet des modifications effectuées
- **MVC** : est un modèle d'architecture logicielle qui permet de séparer les responsabilités dans une application.
- **Cowboy coding** : méthode de développement logiciel où un programmeur écrit du code de manière sans tests formels

WEBIOGRAPHIE

- <https://www.jenkins.io/doc/book/installing/>
- <https://docs.github.com/en/get-started>
- <https://www.redhat.com/fr/topics/devops/what-is-ci-cd>
- <https://www.talend.com/fr/resources/guide-microservices/>
- <https://www.atlassian.com/git/tutorials/git-ssh>
- <https://www.talend.com/fr/resources/monolithic-architecture/>
- <https://loud-technology.com/blog/refactoriser-un-monolithe-en-microservices/>
- <https://www.zabbix.com/documentation/current/en/manual/config/triggers/trigger>
- <https://fr.wikipedia.org/wiki/Mod%C3%A8le-vue-contr%C3%B4leur>
- <https://openclassrooms.com/fr/courses/6173501-apprenez-a-programmer-en-java?archived-source=26832>
- <https://tech-computer.fr/blog/guide-complet-pour-configurer-un-serveur-avec-succes/>
- <https://spring.io/projects/spring-boot>
- <https://vuejs.org/guide/introduction.html>
- <https://laragon.org/docs/install>
- <https://nodejs.org/en/about>
- <https://fr.react.dev/>
- <https://www.react-spring.dev/>
- <https://www.redhat.com/fr/topics/devops/what-cicd-pipeline>

TABLES DE MATIÈRES

REMERCIEMENTS.....	II
SOMMAIRE	III
LISTE DES TABLEAUX	IV
LISTES DES FIGURES	V
INTRODUCTION GÉNÉRALE	1
PARTIE 1 : PRÉSENTATION DE LA CNPS ET ACCUEIL	3
INTRODUCTION	4
ABSTRACT	5
ACCUEIL ET INSERTION	6
PRESENTATION DES TRAVAUX	13
PHASE 1 : REALISATION D'UNE APPLICATION WEB DE CREATION DE COMPTE	18
DOSSIER D'ANALYSE	19
1. Présentation de l'Architecture de l'application	19
1.1 Architecture logique (3-tiers)	19
1.2 Description de l'architecture logique	20
2. Présentation des diagrammes	22
2.1 Diagramme de cas d'utilisation (usecase)	23
2.2 Diagramme de composants.....	26
2.3 Diagramme de Déploiement.....	27
I. CAPTURES DES INTERFACES DES PAGES DE L'APPLICATION.....	28
1. Page d'accueil et de connexion.....	28
2. Page d'inscription	28
3. Page de modification de profil	29
4. Page Dashboard	29
5. Interface Base de Données.....	30
PHASE 2 : TEST DE PRISE EN MAIN DE JENKINS	32
II. INSTALLATION DE JENKINS ET GIT SOUS LE SERVEUR	33
1. INSTALLATION DE GIT	33

2. INSTALLATION DE JENKINS	33
a. Installation du JDK.....	33
b. Installation de Jenkins	34
c. Justificatif du choix du fichier .war de jenkins et jenkins.msi	34
RECOMMANDATIONS.....	46
JUSTIFICATION DES RECOMMANDATIONS	49
PERSPECTIVES.....	54
PERSPECTIVE DE RESTRUCTURATION DE L'ARCHITECTURE	55
1. Refactorisation des responsabilités et modernisation de l'architecture monolithique	55
1.1 JSP existante	55
1.3 Présentation	57
.....	58
CONCLUSION GENERALE	60
ABREVIATIONS ET MOT-CLES	61
WEBIOGRAPHIE	62
TABLES DE MATIÈRES	63